Chapter 7

Learning Non-Relational Databases with AI Assistance

Table of Contents

Chapter 7 1

Learning Non-Relational Databases with AI Assistance 1

7.1. Overview of Databases 2

7.1.1. What is a Non-Relational Database? 2

7.2. Understangin NoSQL using MongoDB 4

7.3. CRUD 23

7.3.1. What is CRUD in MongoDB? 23

7.3.2. Read Operations 26

7.3.3. Update Operations 28

7.3.4. Delete Operations 30

7.3.5. First MongoDB database 31

7.3.6. MongoDB Operator Types 63

7.3.7. Mongoose Validation 74

References 80

7.1. Overview of Databases

A database is a system used to store, manage, and retrieve data efficiently. Traditional applications often rely on **relational databases**, where data is organized into structured tables with rows and columns. However, as modern applications grow in complexity—especially web apps, mobile apps, and real-time systems—more flexible solutions are needed. This is where **non-relational databases** come in.
Non-relational databases are designed to handle large amounts of unstructured or semi-structured data with high flexibility and scalability.

7.1.1. What is a Non-Relational Database?

A **non-relational database**, also known as a **NoSQL database**, stores data in formats other than traditional tables. Instead of requiring a fixed schema, it allows data to be stored in a more dynamic and flexible way.
This means each record can have different fields, structures, or types of data without breaking the system.

Types of Non-Relational Databases

Non-relational databases come in several forms:
a) Document-Based Databases
These store data in documents, usually in JSON or BSON format. Each document contains key-value pairs.
Example:

```
{
  "name": "Alice",
  "age": 25,
  "skills": ["JavaScript", "Node.js"]
}
```

A popular example is MongoDB.
b) Key-Value Stores
Data is stored as a simple key and value pair, similar to a dictionary.
Example:
`"user123" → "John Doe"`

This type is extremely fast and commonly used for caching.

c) Column-Family Databases

Instead of rows, data is stored in columns grouped into families. This is useful for large-scale analytics and distributed systems.

d) Graph Databases
These store data in nodes and relationships, making them ideal for social networks, recommendation systems, and fraud detection.
Example:

- User A → "friends with" → User B

Advantages of Non-Relational Databases

Non-relational databases offer several benefits:

- **Flexibility**: No fixed schema is required.
- **Scalability**: Easily handles large amounts of data across multiple servers.
- **Performance**: Optimized for fast read/write operations.
- **Adaptability**: Suitable for rapidly changing data structures.

Limitations

Despite their advantages, non-relational databases also have drawbacks:

- Less support for complex joins and relationships
- Weaker consistency in some systems (eventual consistency)
- Not ideal for highly structured data requiring strict rules

When to Use Non-Relational Databases

They are best used in:

- Real-time applications (chat apps, gaming)
- Big data systems
- Content management systems
- Social media platforms
- IoT applications

Relational vs Non-Relational Databases

Feature	Relational Database	Non-Relational Database
Structure	Tables	Documents, key-value, graphs
Schema	Fixed	Flexible
Scalability	Vertical	Horizontal
Best for	Structured data	Large, dynamic data

Non-relational databases are a powerful solution for modern applications that require flexibility, speed, and scalability. While they do not replace relational databases entirely, they complement them in building efficient, large-scale systems.
Understanding when and how to use NoSQL databases is an essential skill for today's developers.

7.2. Understangin NoSQL using MongoDB

MongoDB is a document database. It stores data in a type of JSON
(JavaScript Object Notation) format called BSON. A record in MongoDB is a document, which is a data structure composed of key-value pairs like the structure of JSON objects.

It is categorized under the NoSQL (Not only SQL) database because the storage and retrieval of data in the MongoDB are not in the form of tables.
The MongoDB database is developed and managed by MongoDB.Inc under SSPL(Server Side Public License) and initially released in February 2009. It also provides official driver support for all the popular languages like C, C++, C#, and .Net, Go, Java, Node.js, Perl, PHP, Python, Motor, Ruby, Scala, Swift, Mongoid. So, that you can create an application using any of these languages. Nowadays there are so many companies that used MongoDB like Facebook, Nokia, eBay, Adobe, Google, etc. to store their large amount of data. (Ankita_Saini, What is MongoDB – Working and Features, 2021)

How it works ?

Now, we will see how actually thing happens behind the scenes. As we know that MongoDB is a database server and the data is stored in these databases. Or in other words, MongoDB environment gives you a server that you can start and then create multiple databases on it using MongoDB.
Because of its NoSQL database, the data is stored in the collections and documents. Hence the database, collection, and documents are related to each other as shown below:

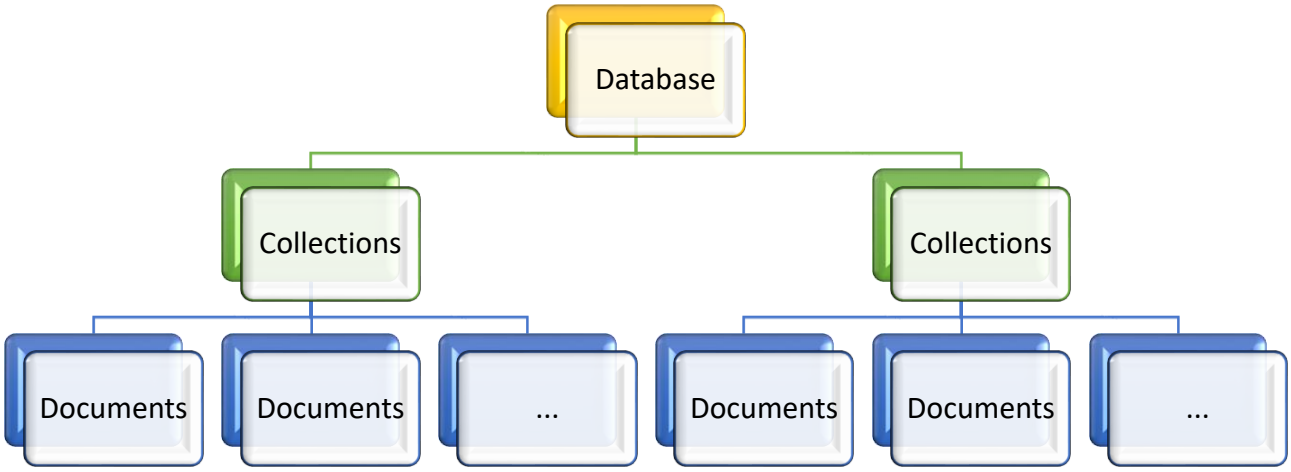


Figure 1 - hierarchical diagram illustrating the role of a NoSQL architect.

- The MongoDB database contains collections just like the MYSQL database contains tables. You are allowed to create multiple databases and multiple collections.
- Now inside of the collection we have documents. These documents contain the data we want to store in the MongoDB database and a single collection can contain multiple documents and you are schema-less means it is not necessary that one document is like another.
- The documents are created using the fields. Fields are key-value pairs in the documents, it is just like columns in the relation database. The value of the fields can be of any BSON data types like double, string, boolean, etc.
- The data stored in the MongoDB is in the format of BSON documents. Here, BSON stands for Binary representation of JSON documents. Or in other words, in the backend, the MongoDB server converts the JSON data into a binary form that is known as BSON and this BSON is stored and queried more efficiently.
- In MongoDB documents, you are allowed to store nested data. This nesting of data allows you to create complex relations between data and store them in the same document which makes the working and fetching of data extremely efficient as compared to SQL. In SQL, you need to write complex joins to get the data from table 1 and table 2. The maximum size of the BSON document is 16MB.

NOTE: In MongoDB server, you are allowed to run multiple databases.
For example, we have a database named GeeksforGeeks. Inside this database, we have two collections and in these collections we have two documents. Now, these documents we store our data in the form of fields. As shown in the below image:

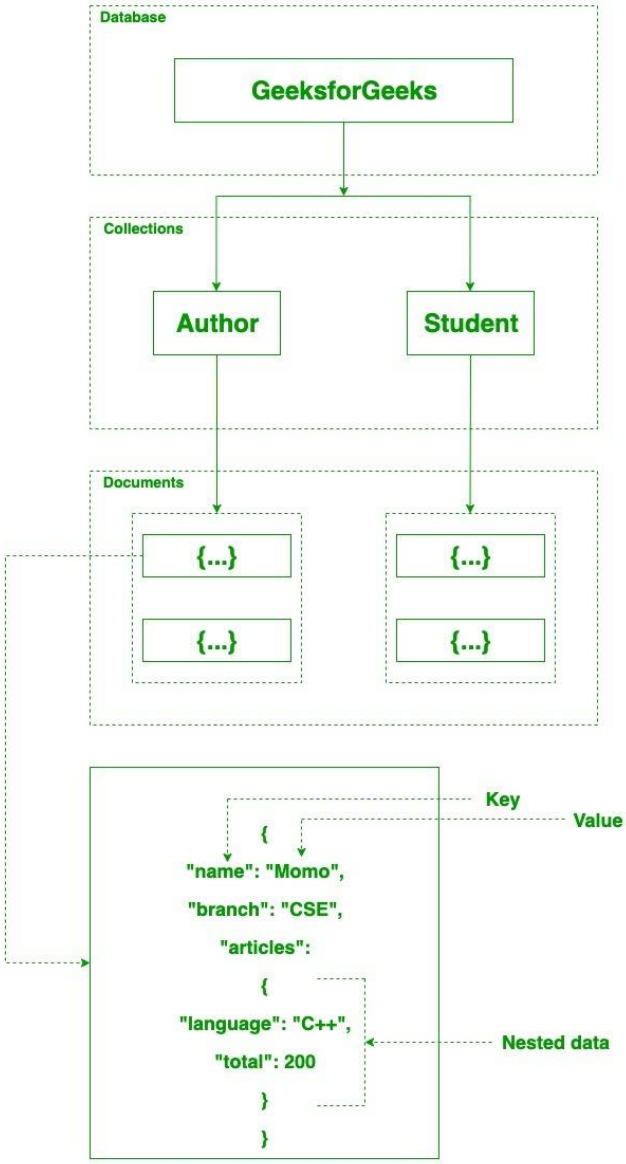


Figure 2- Hierarchy diagram illustrating the architectural structure of MongoDB.

Features of MongoDB –

- **Schema-less Database:** It is the great feature provided by the MongoDB. A Schema-less database means one collection can hold different types of documents in it. Or in other words, in the MongoDB database, a single collection can hold multiple documents and these documents may consist of the different numbers of fields, content, and size. It is not necessary that the one document is similar to another document like in the relational databases. Due to this cool feature, MongoDB provides great flexibility to databases.
- **Document Oriented:** In MongoDB, all the data stored in the documents instead of tables like in RDBMS. In these documents, the data is stored in fields(key-value

pair) instead of rows and columns which make the data much more flexible in comparison to RDBMS. And each document contains its unique object id.

- **Indexing:** In MongoDB database, every field in the documents is indexed with primary and secondary indices this makes easier and takes less time to get or search data from the pool of the data. If the data is not indexed, then database search each document with the specified query which takes lots of time and not so efficient.
- **Scalability:** MongoDB provides horizontal scalability with the help of sharding. Sharding means to distribute data on multiple servers, here a large amount of data is partitioned into data chunks using the shard key, and these data chunks are evenly distributed across shards that reside across many physical servers. It will also add new machines to a running database.
- **Replication:** MongoDB provides high availability and redundancy with the help of replication, it creates multiple copies of the data and sends these copies to a different server so that if one server fails, then the data is retrieved from another server.
- **Aggregation:** It allows to perform operations on the grouped data and get a single result or computed result. It is similar to the SQL GROUPBY clause. It provides three different aggregations i.e, aggregation pipeline, map-reduce function, and single-purpose aggregation methods.
- **High Performance:** The performance of MongoDB is very high and data persistence as compared to another database due to its features like scalability, indexing, replication, etc.

Advantages of MongoDB

- It is a schema-less NoSQL database. You need not to design the schema of the database when you are working with MongoDB.
- It does not support join operation.
- It provides great flexibility to the fields in the documents.
- It contains heterogeneous data.
- It provides high performance, availability, scalability.
- It supports Geospatial efficiently.
- It is a document oriented database and the data is stored in BSON documents.
- It also supports multiple document ACID transition(string from MongoDB 4.0).
- It does not require any SQL injection.
- It is easily integrated with Big Data Hadoop

Disadvantages of MongoDB

- It uses high memory for data storage.

- You are not allowed to store more than 16MB data in the documents.
- The nesting of data in BSON is also limited you are not allowed to nest data more than 100 levels.

How mongoDB is different from RDBMS ?

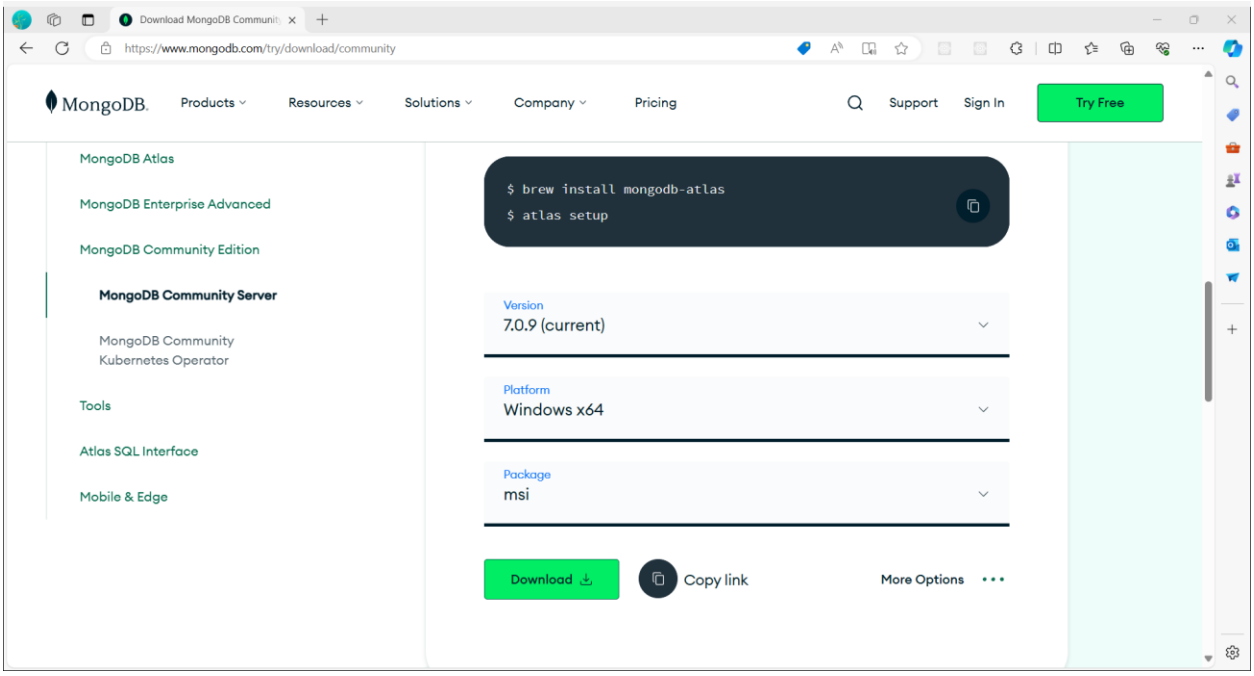
Some major differences in between MongoDB and the RDBMS are as follows:

MongoDB	RDBMS
It is a non-relational and document-oriented database.	It is a relational database.
It is suitable for hierarchical data storage.	It is not suitable for hierarchical data storage.
It has a dynamic schema.	It has a predefined schema.
It centers around the CAP theorem (Consistency, Availability, and Partition tolerance).	It centers around ACID properties (Atomicity, Consistency, Isolation, and Durability).
In terms of performance, it is much faster than RDBMS.	In terms of performance, it is slower than MongoDB.

Installing MongoDB in Windows

To install MongoDB in Windows:

1. go to Install MongoDB Community Edition on Windows — https://www.mongodb.com/try/download/community
2. In the MongoDB community server download, scroll down until you see a green ***Download*** button



3. Click the 'Version' dropdown button to check on the different version of MongoDB. For this class project, we will use 'Version 7.0.9.'
4. Click on the 'Platform' dropdown button to check and pick the platform that we work on. If you are working on the Windows, select the 'Windows x64'.
5. After the 'Version' and 'Platform' are selected, click on the ***Download*** button.
6. After the installer is completely downloaded, go the 'Download' folder and double click on it 'Windows Installer Package' to install MongoDB:

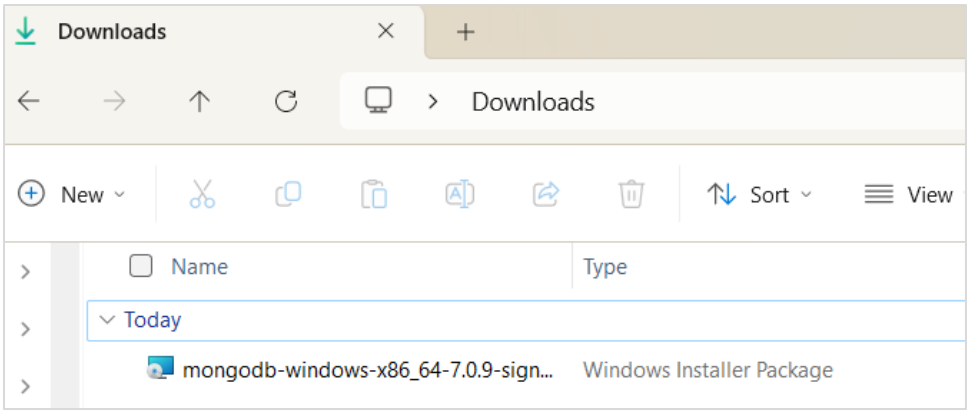


Figure 3 - MongoDB Windows Installer Package in the local computer Download folder.

7. After double-clicking on the 'Windows Installer Package', the MongoDB installation window will pop-up:

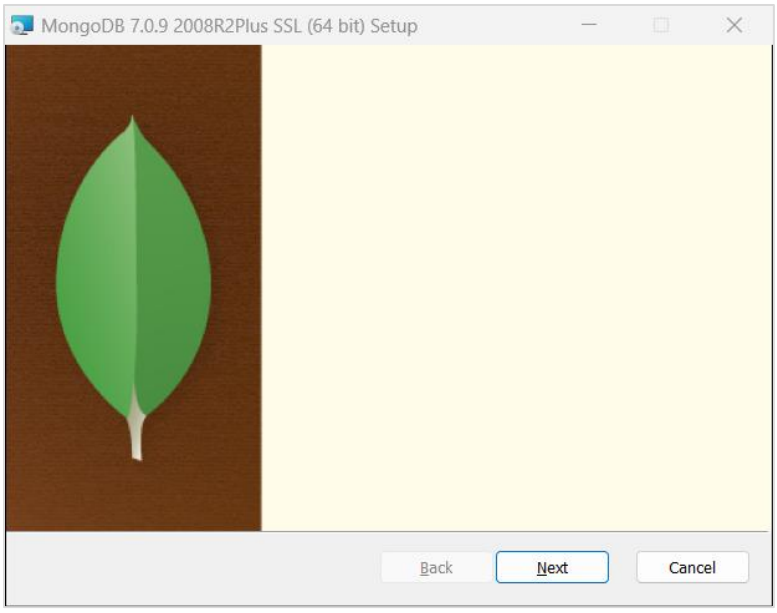


Figure 4 - introduction to MongoDB 7.0.9 2008R2Plus SSL (64 bit) Setup window

8. Press **Next.** In the next window, check on the **I accept the terms in the License Agreement** checkbox and press **Next:**

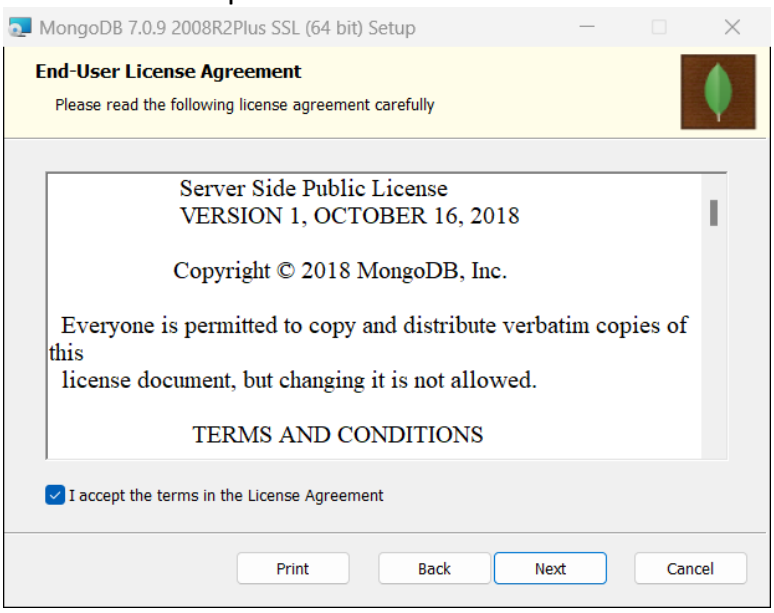


Figure 5 - 'End-User License Agreement' MongoDB 7.0.9 2008R2Plus SSL (64 bit) Setup window

9. In the 'Choose Setup Type', click on the **Complete** button to select the setup type:

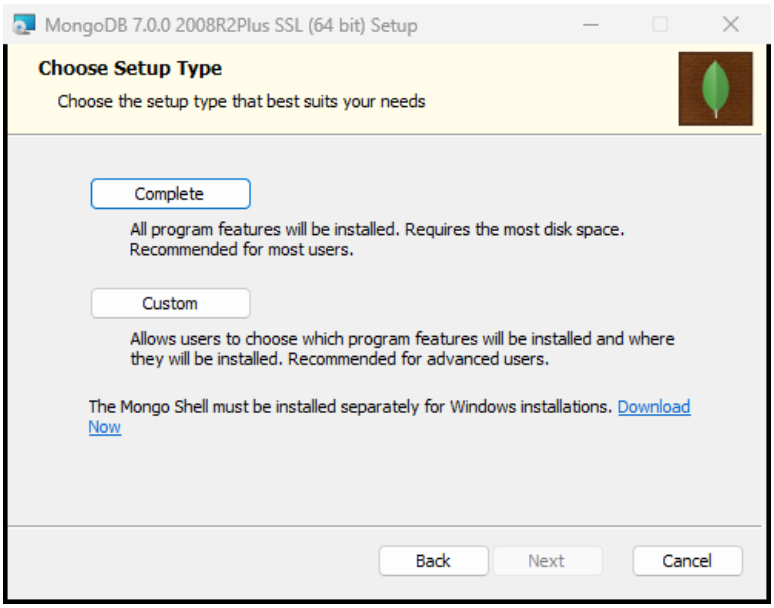


Figure 6 - 'Choose Setup Type' MongoDB 7.0.9 2008R2Plus SSL (64 bit) Setup window

10. In the 'Service Configuration' window, make sure that the **Install MongoDB as Service** is checked and **Run service as Network Service user** is selected. Also take close look on the location path of the **Data Directory** and **Log Directory,** these are locations where your MongoDB data will be saved. **P**ress **Next** to continue :

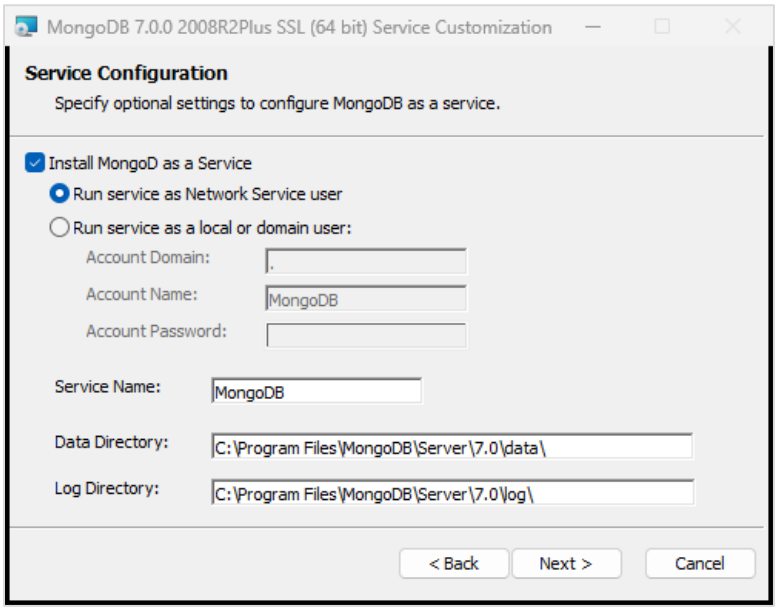


Figure 7 - 'Service Configuration' MongoDB 7.0.0 2008R2Plus SSL (64 bit) Service Customization window

(chatgpt, Question: what does 'run service as network service user' mean?, 2024)When you install MongoDB, you're often given the option to specify which user account the MongoDB service should run under on your system. "Run service as Network Service

user" typically means running the MongoDB service under the built-in Windows account called "Network Service."
The "Network Service" account is a special built-in account that has fewer privileges than the Local System account but more permissions than a standard user account. It's commonly used for services that need network access but don't require extensive system permissions.
Choosing to run the MongoDB service as the Network Service user can be a security measure, as it restricts the permissions of the MongoDB service to minimize potential security vulnerabilities. It's generally a good practice to run services with the least privileges necessary to perform their required tasks.

11. In the 'Ready to install MongoDB 7.0.0 2008R2Plus SSL(64 bit) window, complete the installation by clicking on the **Install** button:

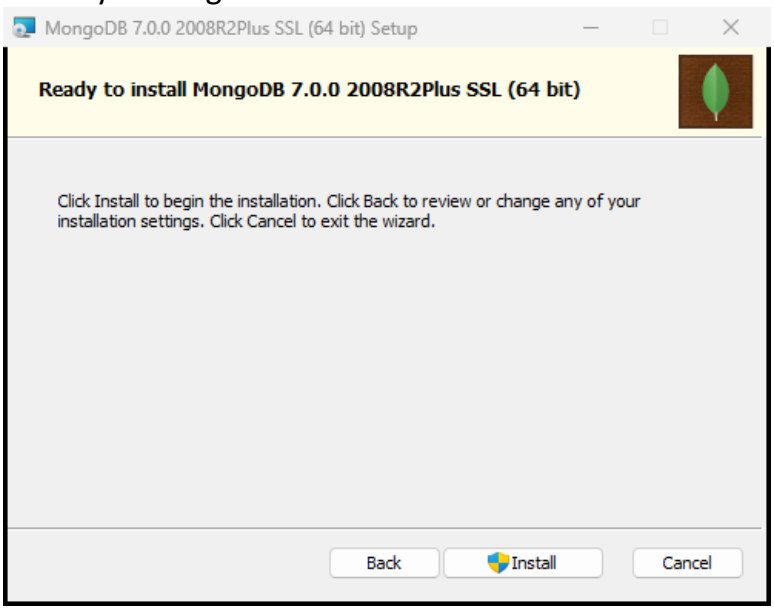
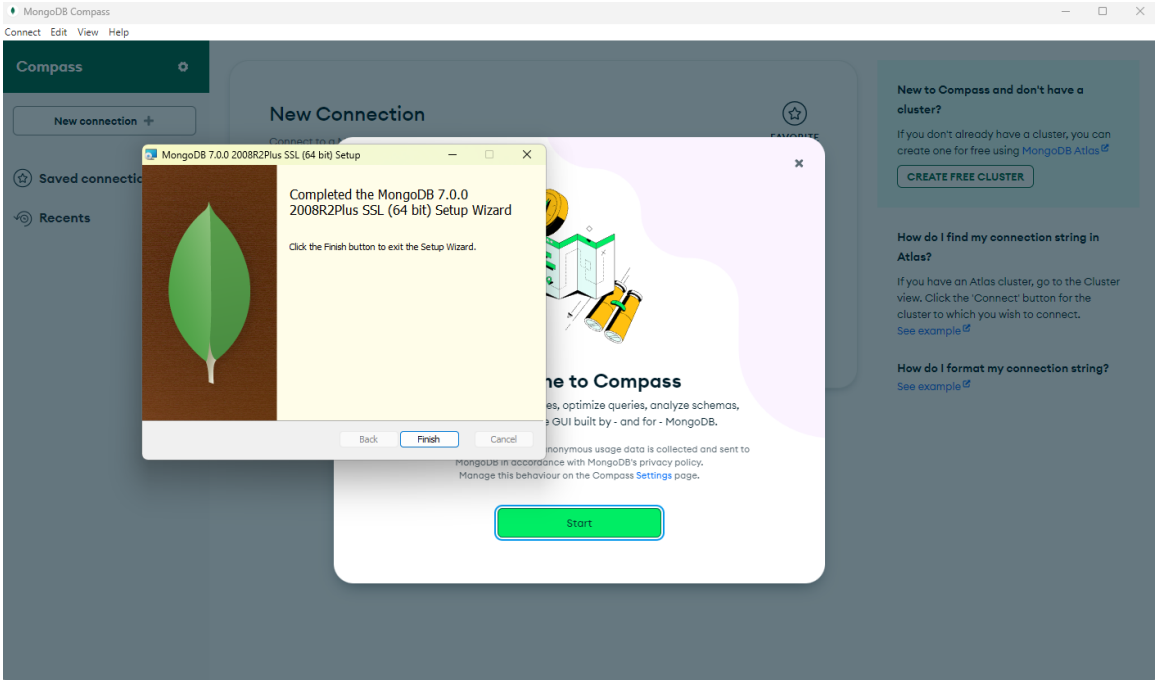


Figure 8 - 'Ready to install MongoDB' MongoDB 7.0.0 2008R2Plus SSL (64 bit) Service Customization window

12. Once the installation is complete, MongoDB compress window and the completed installation window open:



13. You can click the **Start** button in the 'Welcome to Compass' to explore a little bit about Compass. We can see that MongoDB already generates a URL as:

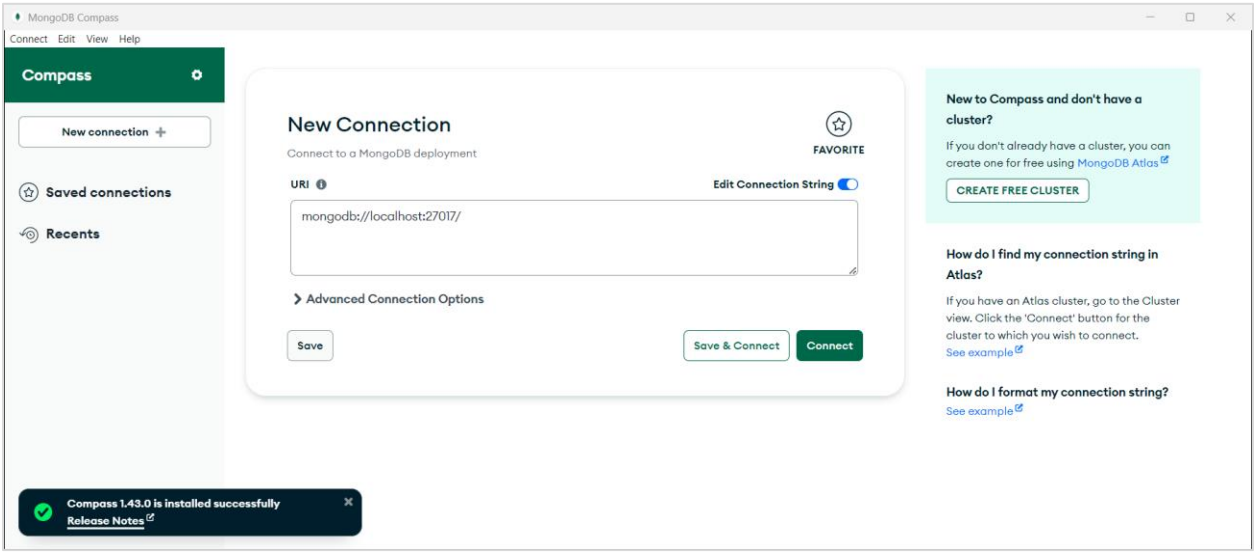


Figure 9 - MongoDB Compass

14. The **URL** link connects us to our MongoDB database. From Compress, we just need to click the **Connect** button to connect to MongoDB database:

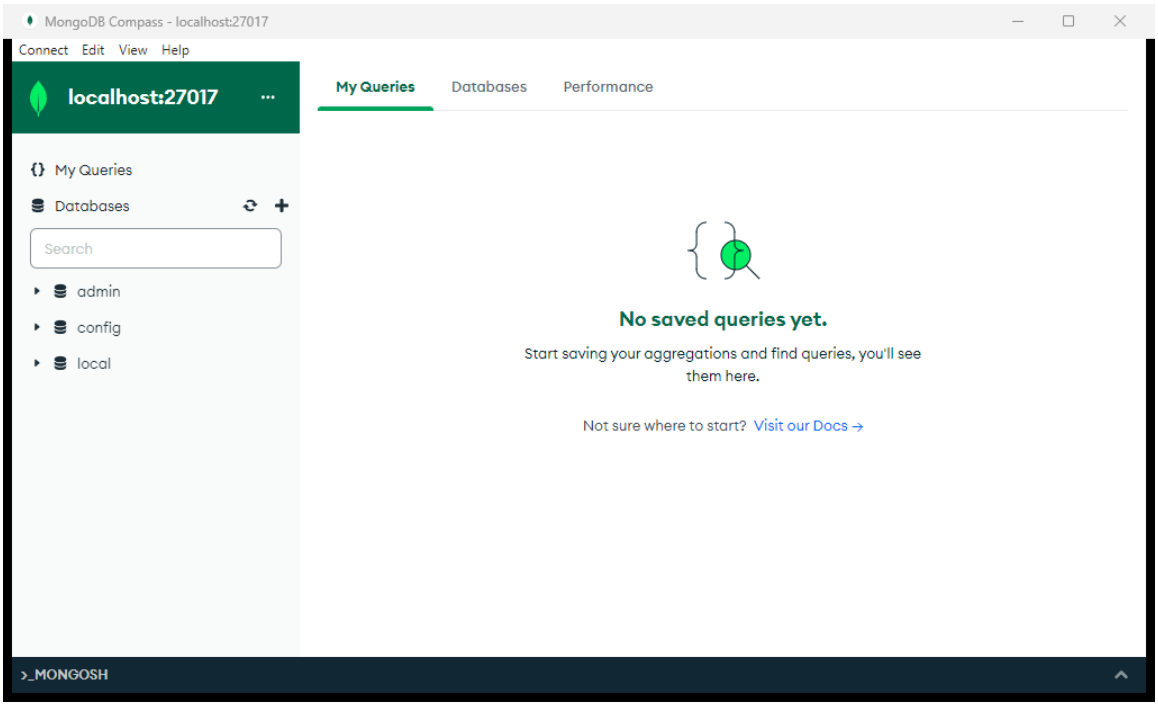
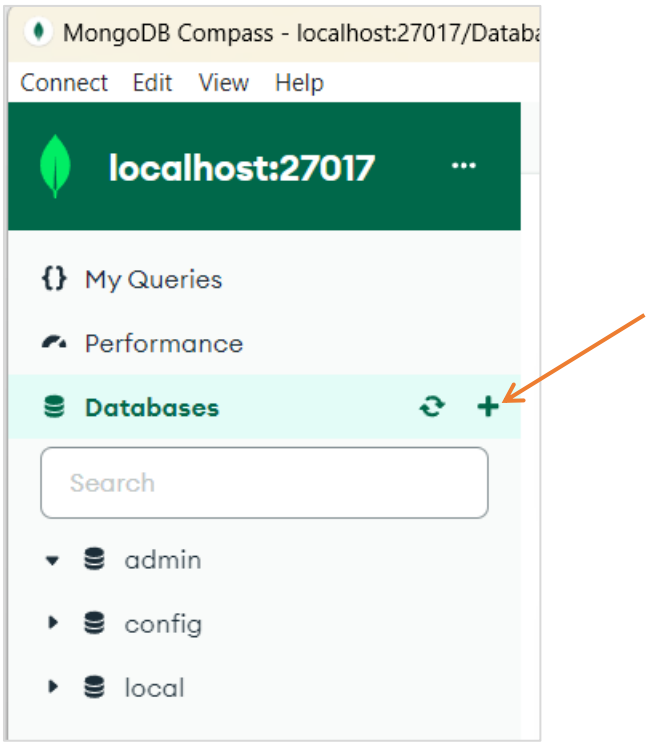


Figure 10 - MongoDB Compass database

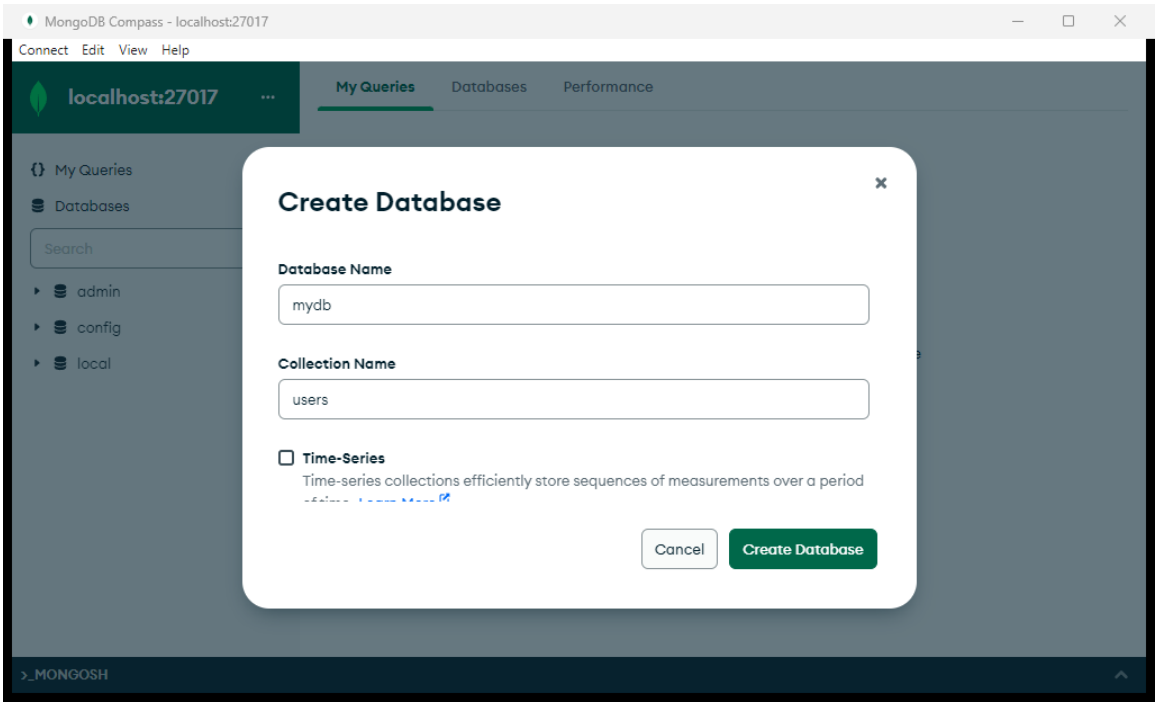
15. MongoDB has some database already created (***admin, config, local)*** (chatgpt, chatgpt, 2024) In MongoDB, the admin database, the config database, and the local database serve different purposes:
    - **admin Database:** This database is crucial for administrative tasks. It typically stores user authentication information and authorization data. You can manage user roles and permissions, configure authentication mechanisms, and perform administrative operations such as creating backups and managing replica sets or sharded clusters.
    - **config Database:** This database is specifically used in MongoDB sharding. Sharding is a method for distributing data across multiple machines. The config database stores metadata about the sharded cluster configuration, including information about the sharded collections, chunks, and the mapping between shard keys and chunks.
    - **local Database:** This database is automatically created on each MongoDB instance. It's primarily used for internal purposes, including storing replica set configuration information, as well as any database-specific collections like oplog.rs, which is crucial for replication and maintaining a consistent state across replica sets.

    These databases are part of MongoDB's internal infrastructure and play vital roles in its functioning, especially in distributed environments and administrative operations.

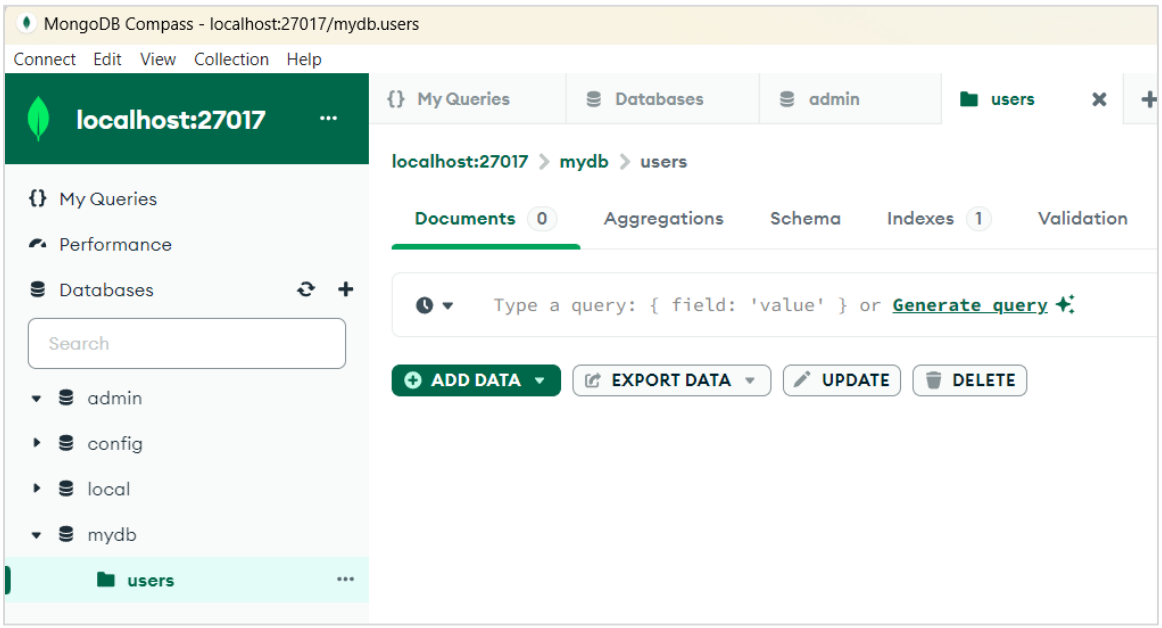
16. You can create your own database, in the left column, next to the right of the link ‘Databases’ click on the plus sign



Activity: create a new database named 'mydb' with a collection named 'users.' After inputting the necessary details, press 'Create Database' to initiate the creation process.



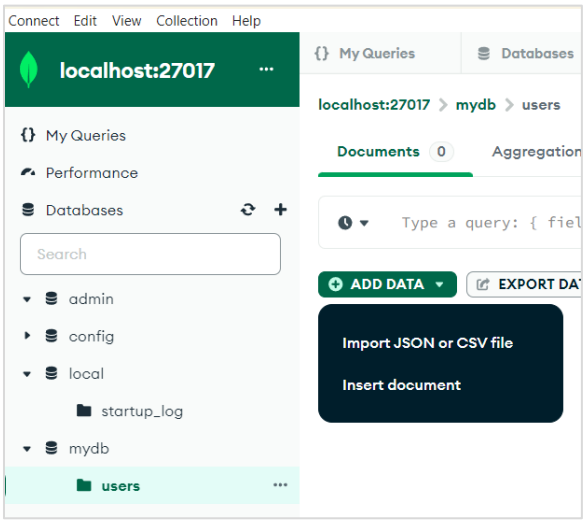
Have a screenshot of the MongoDB Compass *clearly* showing the new database ***mydb***:



We can import data to our database using the **ADD DATA** button, or to export data by clicking the **EXPORT DATA** button.

17. To insert data into a database, we click on the database, which is listed on the left column. Once the database opens the list of collections, we click on the collection

which we want to add data. In the collection workspace, click on the button that says **ADD DATA.** After clicking on 'ADD DATA' a dropdown list appears:



From the dropdown list, if we want to type our data, we click on 'Insert document'. On the other hand, if we want to import a JSON or CSV file, we click on 'Import JSON or CSV file'

Activity: Let's add data to collection 'users' in our database 'mydb'. Following the previous step to open an 'Insert Document' window. In the 'Insert Document' window, type two new key:pair to the collection as:

"

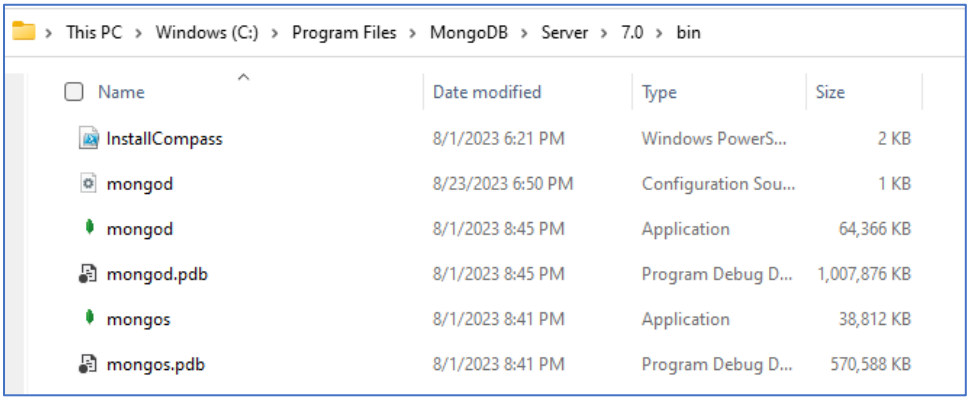
```
{
  "_id": {
    "name": "student's full name",
    "email": "student's email address",
  }
}
```

Have a screenshot of the 'Insert Document' with the "name" and "email" keys. Type your full name and email address as the key's values, respectively. Paste the screenshot in the space below:

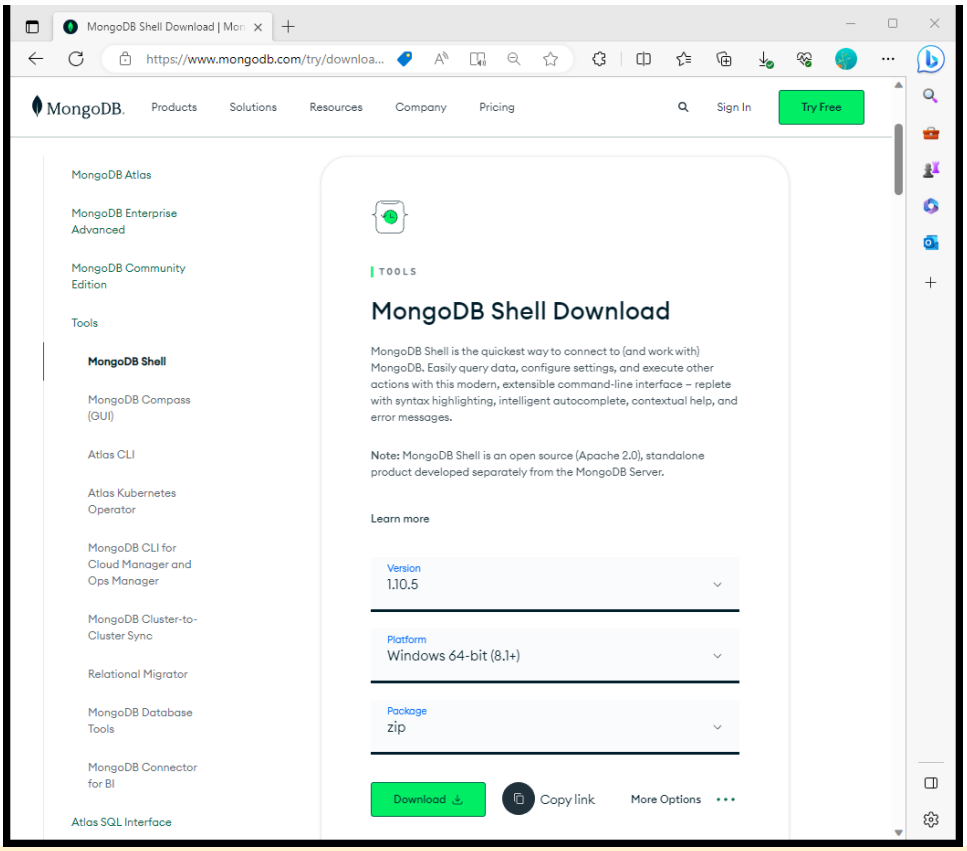
18. Close MongoDB.

Activity: After closing MongoDB, you can reopen it by clicking on the icon that was automatically placed on your computer desktop upon installation. Provide a screenshot of your computer desktop displaying the MongoDB icon to confirm installation. Paste the screenshot in the space below:

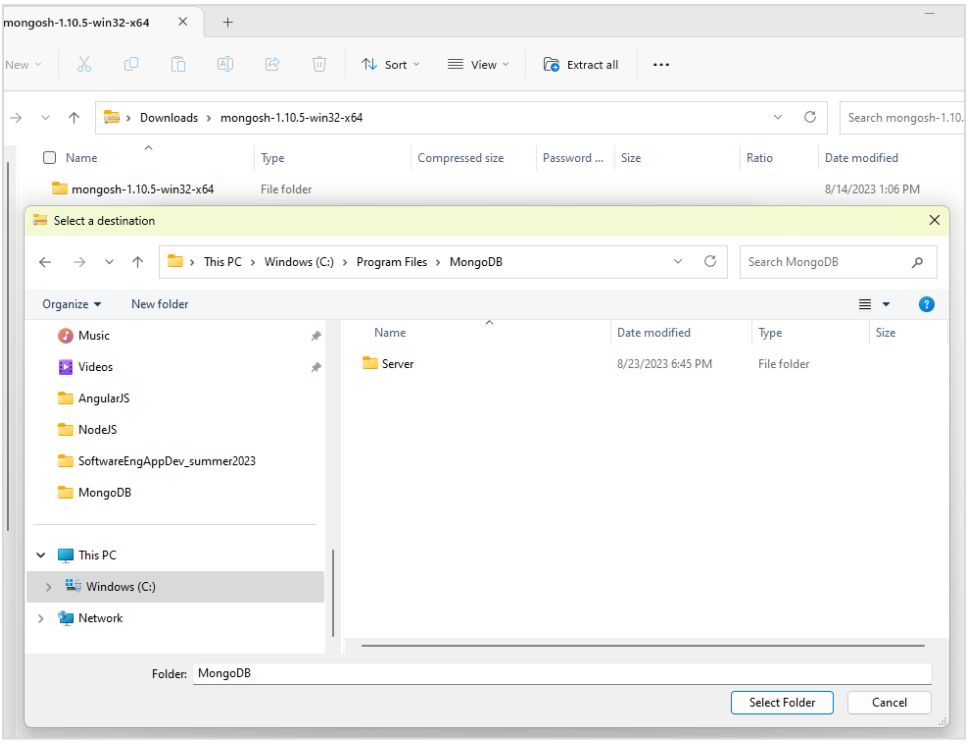
19. Let's explore the MongoDB folders. Go to the location of the MongoDB folder, and double-click on the 'Bin' folder. In the **bin** folder contains the MongoDB utility as shown below:



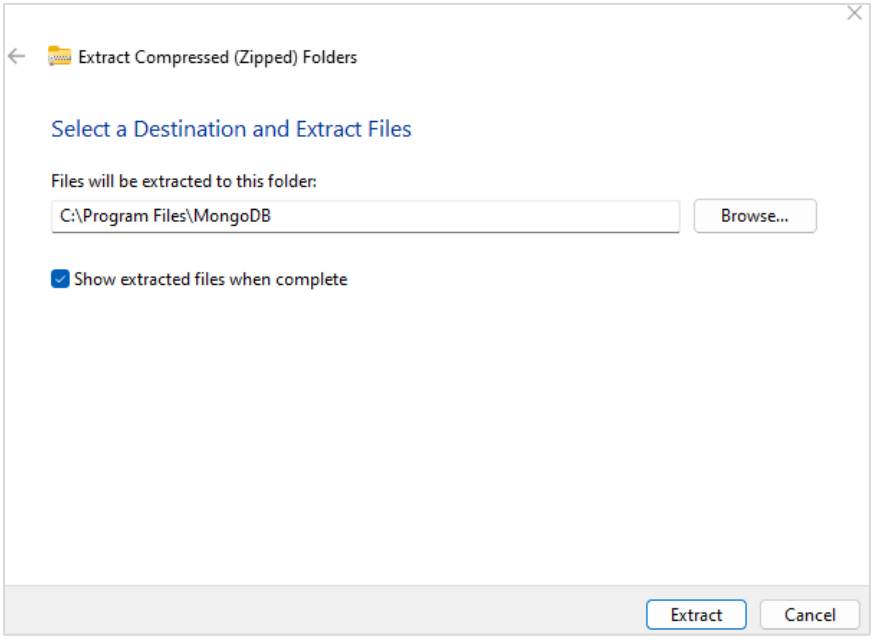
20. If you want to run MongoDB from the Terminal, you will need to download the MongoDB shell for Windows' users. To download MongoDB shell, visit: https://www.mongodb.com/try/download/shell



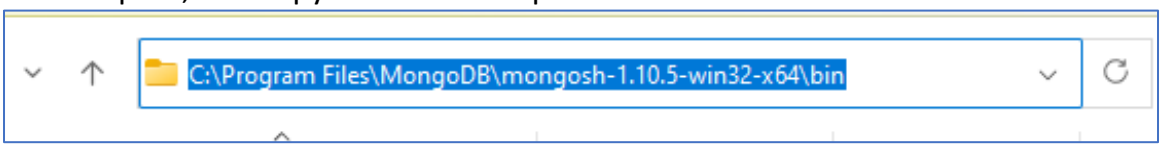
It is very important that during the installation process, install MongoDB shell in the MongoDB folder as shown below



21. After selecting the location, extract the content in the folder by clicking on the 'Extract button:

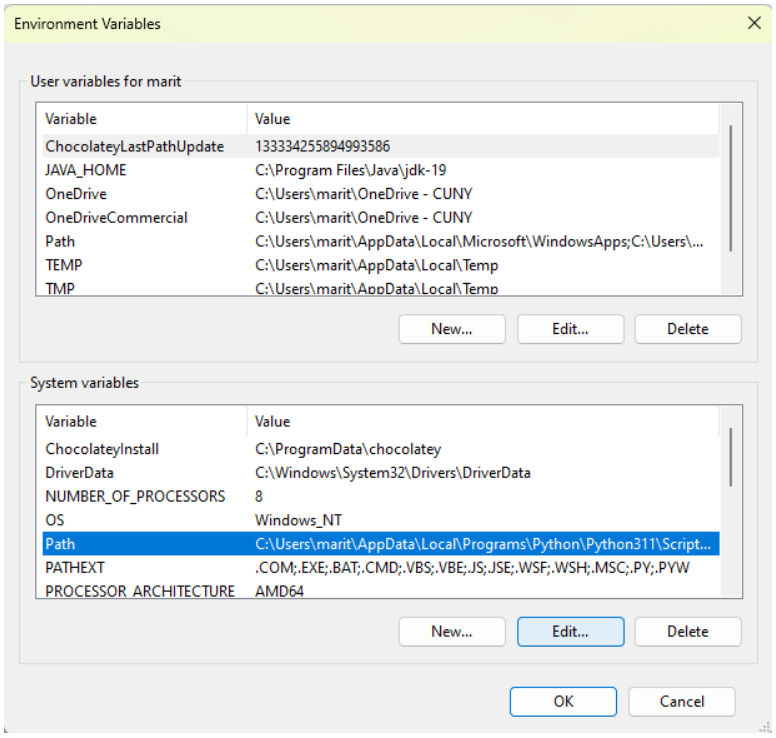


22. After the folder is completely extracted, navigate to the *bin* folder, click on the address path, and copy the *bin* folder path.

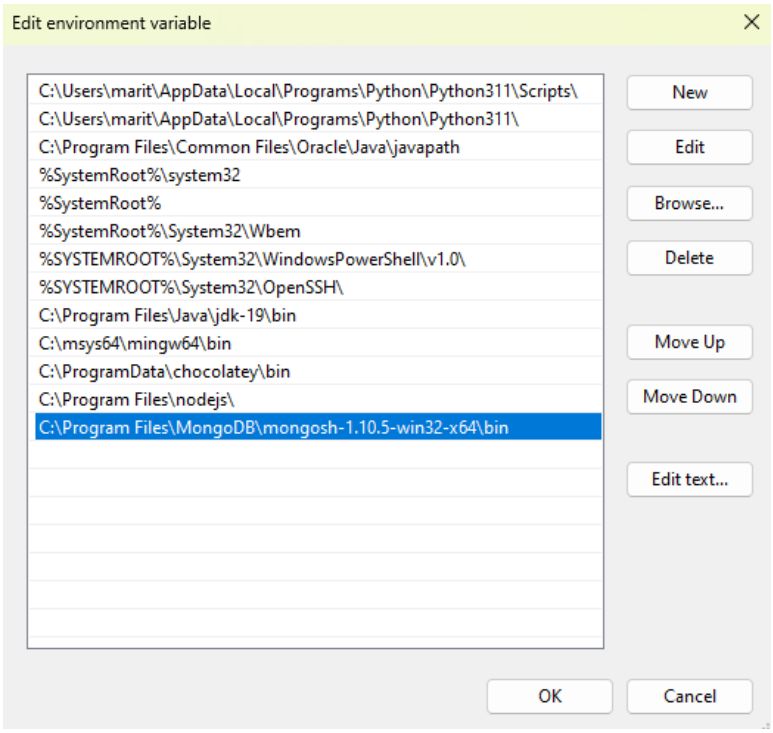


23. With the copied path, go to system variable to add the path to the computer system's variables. To do this:

- open the *System Properties* window by typing 'System environment variables' in the Windows' Search engine.
- In the *System Properties* window, click on the button that says 'Environment Variables'.
- In the *Environment Variables,* under *System variables,* select '*Path'* and click on *Edit* button*:*



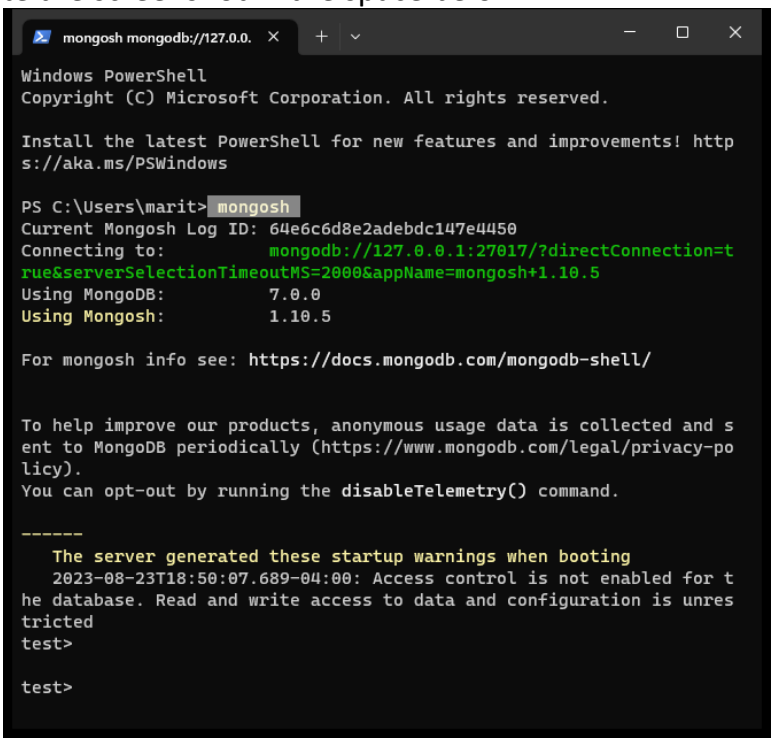
24. In the *Edit Environment Variable* window, click on *New,* and paste the copy path :



Click *OK* when is done.

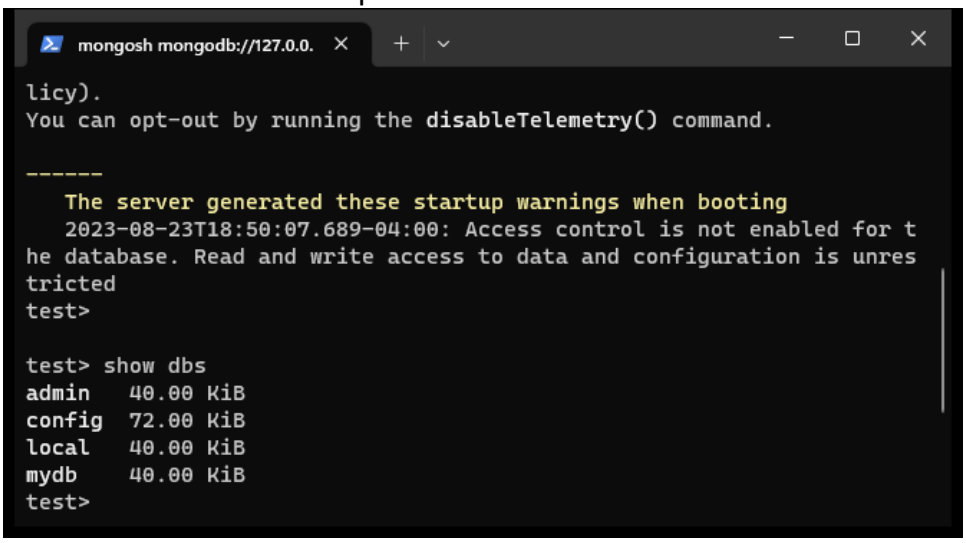
25. To test if the MongoDB is properly linked to the system's variables, open the Terminal, command prompt, and type ***mongosh***

Activity: have a screenshot of the Terminal showing the result after typing ***mongosh.*** Paste the screenshot in the space below:



26. In the Terminal, we can write ***show dbs*** to see the databases that are our MongoDB.

Activity: have a screenshot of the Terminal showing the result after typing ***show dbs.*** Paste the screenshot in the space below:



---------- Installation ends here ----------

Supporting videos:

- How to install mongoDB and compass on mac os https://youtu.be/gB6WLkSrtJk
- How to install mongo DB and compass on mac os - YouTube https://www.youtube.com/watch?v=QfTGtPKfaW4

7.3. CRUD

7.3.1. What is CRUD in MongoDB?

CRUD operations describe the conventions of a user-interface that let users view, search, and modify parts of the database.

MongoDB documents are modified by connecting to a server, querying the proper documents, and then changing the setting properties before sending the data back to the database to be updated. CRUD is data-oriented, and it's standardized according to HTTP action verbs.

When it comes to the individual CRUD operations:

1. The **C**reate operation is used to insert new documents in the MongoDB database.

2. The **R**ead operation is used to query a document in the database.
3. The **U**pdate operation is used to modify existing documents in the database.
4. The **D**elete operation is used to remove documents in the database.

How to Perform CRUD Operations

Now that we've defined MongoDB CRUD operations, we can take a look at how to carry out the individual operations and manipulate documents in a MongoDB database. Let's go into the processes of creating, reading, updating, and deleting documents, looking at each operation in turn.

Create Operations

For MongoDB CRUD, if the specified collection doesn't exist, the create operation will create the collection when it's executed. Create operations in MongoDB target a single collection, not multiple collections. Insert operations in MongoDB are atomic on a single document level.
MongoDB provides two different create operations that you can use to insert documents into a collection:

- `db.collection.insertOne()`
- `db.collection.insertMany()`

insertOne()

As the namesake, insertOne() allows you to insert one document into the collection. For this example, we're going to work with a collection called RecordsDB. We can insert a single entry into our collection by calling the insertOne() method on RecordsDB. We then provide the information we want to insert in the form of key-value pairs, establishing the schema.

```
db.RecordsDB.insertOne({
    name: "Marsh",
    age: "6 years",
    species: "Dog",
    ownerAddress: "380 W. Fir Ave",
    chipped: true
})
```

If the create operation is successful, a new document is created. The function will return an object where “acknowledged” is “true” and “insertID” is the newly created “ObjectId.”

```
> db.RecordsDB.insertOne({
... name: "Marsh",
... age: "6 years",
... species: "Dog",
... ownerAddress: "380 W. Fir Ave",
... chipped: true
... })
{
        "acknowledged" : true,
        "insertedId" : ObjectId("5fd989674e6b9ceb8665c57d")
}
```

insertMany()

It’s possible to insert multiple items at one time by calling the *insertMany()* method on the desired collection. In this case, we pass multiple items into our chosen collection (*RecordsDB*) and separate them by commas. Within the parentheses, we use brackets to indicate that we are passing in a list of multiple entries. This is commonly referred to as a nested method.

```
db.RecordsDB.insertMany([{
    name: "Marsh",
    age: "6 years",
    species: "Dog",
    ownerAddress: "380 W. Fir Ave",
    chipped: true
},
    {name: "Kitana",
     age: "4 years",
     species: "Cat",
     ownerAddress: "521 E. Cortland",
     chipped: true
}])
```

```
db.RecordsDB.insertMany([{ name: "Marsh", age: "6 years",
species: "Dog",
ownerAddress: "380 W. Fir Ave", chipped: true}, {name: "Kitana",
age: "4 years",
species: "Cat", ownerAddress: "521 E. Cortland", chipped: true}])
{
        "acknowledged" : true,
        "insertedIds" : [
                ObjectId("5fd98ea9ce6e8850d88270b4"),
                ObjectId("5fd98ea9ce6e8850d88270b5")
        ]
}
```

7.3.2. Read Operations

The read operations allow you to supply special query filters and criteria that let you specify which documents you want. The MongoDB documentation contains more information on the available query filters. Query modifiers may also be used to change how many results are returned.
MongoDB has two methods of reading documents from a collection:

- db.collection.find()
- db.collection.findOne()

find()

In order to get all the documents from a collection, we can simply use the *find()* method on our chosen collection. Executing just the *find()* method with no arguments will return all records currently in the collection.

```
db.RecordsDB.find()
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" :
"Kitana", "age" : "4 years", "species" : "Cat",
"ownerAddress" : "521 E. Cortland", "chipped" : true }
{ "_id" : ObjectId("5fd993a2ce6e8850d88270b7"), "name" :
"Marsh", "age" : "6 years", "species" : "Dog",
"ownerAddress" : "380 W. Fir Ave", "chipped" : true }
```

```
{ "_id" : ObjectId("5fd993f3ce6e8850d88270b8"), "name" :
"Loo", "age" : "3 years", "species" : "Dog", "ownerAddress"
: "380 W. Fir Ave", "chipped" : true }
{ "_id" : ObjectId("5fd994efce6e8850d88270ba"), "name" :
"Kevin", "age" : "8 years", "species" : "Dog",
"ownerAddress" : "900 W. Wood Way", "chipped" : true }
```

Here we can see that every record has an assigned "ObjectId" mapped to the "_id" key. If you want to get more specific with a read operation and find a desired subsection of the records, you can use the previously mentioned filtering criteria to choose what results should be returned. One of the most common ways of filtering the results is to search by value.

```
db.RecordsDB.find({"species":"Cat"})
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" :
"Kitana", "age" : "4 years", "species" : "Cat",
"ownerAddress" : "521 E. Cortland", "chipped" : true }
```

findOne()

In order to get one document that satisfies the search criteria, we can simply use the *findOne()* method on our chosen collection. If multiple documents satisfy the query, this method returns the first document according to the natural order which reflects the order of documents on the disk. If no documents satisfy the search criteria, the function returns null. The function takes the following form of syntax.

```
db.{collection}.findOne({query}, {projection})
```

Let's take the following collection—say, *RecordsDB*, as an example.

```
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" :
"Kitana", "age" : "8 years", "species" : "Cat", "ownerAddress" :
"521 E. Cortland", "chipped" : true }
{ "_id" : ObjectId("5fd993a2ce6e8850d88270b7"), "name" : "Marsh",
"age" : "6 years", "species" : "Dog", "ownerAddress" : "380 W.
Fir Ave", "chipped" : true }
{ "_id" : ObjectId("5fd993f3ce6e8850d88270b8"), "name" : "Loo",
"age" : "3 years", "species" : "Dog", "ownerAddress" : "380 W.
Fir Ave", "chipped" : true }
```

```
{ "_id" : ObjectId("5fd994efce6e8850d88270ba"), "name" : "Kevin",
"age" : "8 years", "species" : "Dog", "ownerAddress" : "900 W.
Wood Way", "chipped" : true }
```

And, we run the following line of code:

```
db.RecordsDB.find({"age":"8 years"})
```

We would get the following result:

```
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" :
"Kitana", "age" : "8 years", "species" : "Cat", "ownerAddress" :
"521 E. Cortland", "chipped" : true }
```

Notice that even though two documents meet the search criteria, only the first document that matches the search condition is returned.

7.3.3. Update Operations

Like create operations, update operations operate on a single collection, and they are atomic at a single document level. An update operation takes filters and criteria to select the documents you want to update.

You should be careful when updating documents, as updates are permanent and can't be rolled back. This applies to delete operations as well.
For MongoDB CRUD, there are three different methods of updating documents:

- db.collection.updateOne()
- db.collection.updateMany()
- db.collection.replaceOne()

updateOne()

We can update a currently existing record and change a single document with an update operation. To do this, we use the *updateOne()* method on a chosen collection, which here is "RecordsDB." To update a document, we provide the method with two arguments: an update filter and an update action.

The update filter defines which items we want to update, and the update action defines how to update those items. We first pass in the update filter. Then, we use the "$set"

key and provide the fields we want to update as a value. This method will update the first record that matches the provided filter.

```
db.RecordsDB.updateOne({name: "Marsh"}, {$set:{ownerAddress: "451
W. Coffee St. A204"}})
{"acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
{"_id" : ObjectId("5fd993a2ce6e8850d88270b7"), "name" : "Marsh",
"age" : "6 years", "species" : "Dog", "ownerAddress" : "451 W.
Coffee St. A204", "chipped" : true }
```

updateMany()

updateMany() allows us to update multiple items by passing in a list of items, just as we did when inserting multiple items. This update operation uses the same syntax for updating a single document.

```
db.RecordsDB.updateMany({species:"Dog"}, {$set: {age: "5"}})
{ "acknowledged" : true, "matchedCount" : 3, "modifiedCount" : 3 }
> db.RecordsDB.find()
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" : "Kitana",
"age" : "4 years", "species" : "Cat", "ownerAddress" : "521 E.
Cortland", "chipped" : true }
{ "_id" : ObjectId("5fd993a2ce6e8850d88270b7"), "name" : "Marsh",
"age" : "5", "species" : "Dog", "ownerAddress" : "451 W. Coffee St.
A204", "chipped" : true }
{ "_id" : ObjectId("5fd993f3ce6e8850d88270b8"), "name" : "Loo", "age"
: "5", "species" : "Dog", "ownerAddress" : "380 W. Fir Ave", "chipped"
: true }
{ "_id" : ObjectId("5fd994efce6e8850d88270ba"), "name" : "Kevin",
"age" : "5", "species" : "Dog", "ownerAddress" : "900 W. Wood Way",
"chipped" : true }
```

replaceOne()

The replaceOne() method is used to replace a single document in the specified collection. replaceOne() replaces the entire document, meaning fields in the old document not contained in the new will be lost.

```
db.RecordsDB.replaceOne({name: "Kevin"}, {name: "Maki"})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
> db.RecordsDB.find()
```

```
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" : "Kitana",
"age" : "4 years", "species" : "Cat", "ownerAddress" : "521 E.
Cortland", "chipped" : true }
{ "_id" : ObjectId("5fd993a2ce6e8850d88270b7"), "name" : "Marsh",
"age" : "5", "species" : "Dog", "ownerAddress" : "451 W. Coffee St.
A204", "chipped" : true }
{ "_id" : ObjectId("5fd993f3ce6e8850d88270b8"), "name" : "Loo", "age"
: "5", "species" : "Dog", "ownerAddress" : "380 W. Fir Ave", "chipped"
: true }
{ "_id" : ObjectId("5fd994efce6e8850d88270ba"), "name" : "Maki" }
```

7.3.4. Delete Operations

Delete operations operate on a single collection, like update and create operations. Delete operations are also atomic for a single document. You can provide delete operations with filters and criteria in order to specify which documents you would like to delete from a collection. The filter options rely on the same syntax that read operations utilize.
MongoDB has two different methods of deleting records from a collection:

- db.collection.deleteOne()
- db.collection.deleteMany()

deleteOne()

deleteOne() is used to remove a document from a specified collection on the MongoDB server. A filter criteria is used to specify the item to delete. It deletes the first record that matches the provided filter.

```
db.RecordsDB.deleteOne({name:"Maki"})
{ "acknowledged" : true, "deletedCount" : 1 }
> db.RecordsDB.find()
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" :
"Kitana", "age" : "4 years", "species" : "Cat", "ownerAddress" :
"521 E. Cortland", "chipped" : true }
{ "_id" : ObjectId("5fd993a2ce6e8850d88270b7"), "name" : "Marsh",
"age" : "5", "species" : "Dog", "ownerAddress" : "451 W. Coffee
St. A204", "chipped" : true }
```

```
{ "_id" : ObjectId("5fd993f3ce6e8850d88270b8"), "name" : "Loo",
"age" : "5", "species" : "Dog", "ownerAddress" : "380 W. Fir
Ave", "chipped" : true }
```

deleteMany()

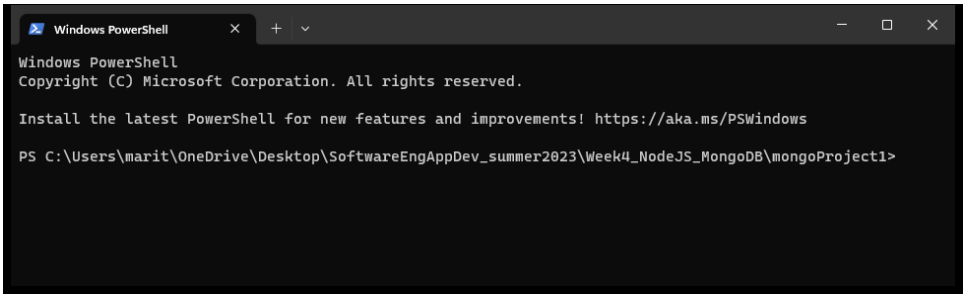
deleteMany() is a method used to delete multiple documents from a desired collection with a single delete operation. A list is passed into the method and the individual items are defined with filter criteria as in *deleteOne()*.

```
db.RecordsDB.deleteMany({species:"Dog"})
{ "acknowledged" : true, "deletedCount" : 2 }
> db.RecordsDB.find()
{ "_id" : ObjectId("5fd98ea9ce6e8850d88270b5"), "name" :
"Kitana", "age" : "4 years", "species" :
```

(MongoDB CRUD Operations, n.d.)

7.3.5. First MongoDB database

To create our first database with MongoDB, we can create a project folder as *mongoProject1* in the Desktop and open the folder in the Terminal:

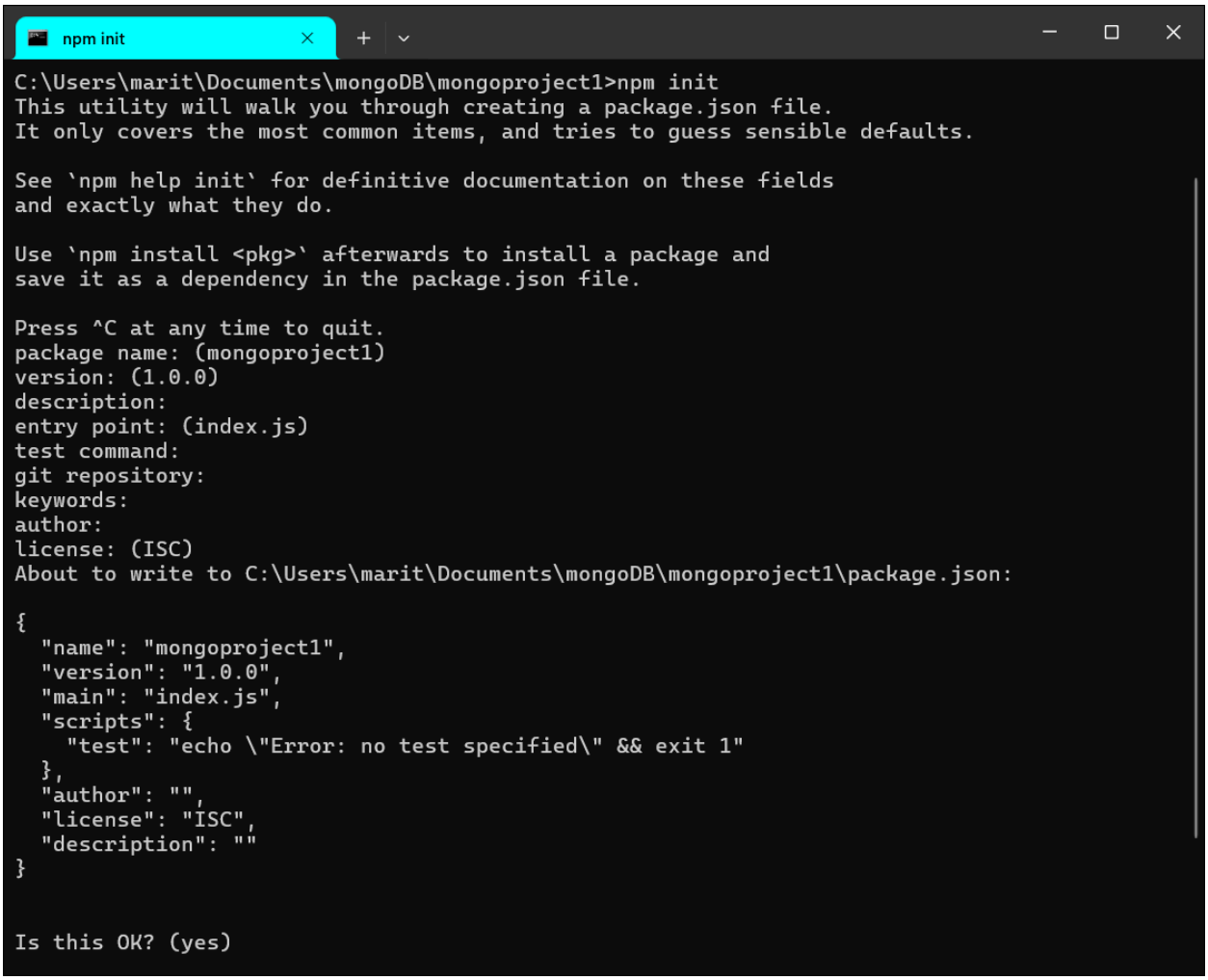


In the project folder, we always need to initialize node package by typing ***npm init.*** After you entered ***npm init***, the Terminal will show a set of question to create the MongoDB database:

- package name
- version
- description (empty)
- entry point

- test command
- git repository
- keywords
- author
- license

<u>we are not changing anything,</u> we will edit those information during the project. Therefore, for all those questions, we just press 'Enter' to confirm. At the end, it will show a JSON structure and we *yes* to confirm:



Later, we are going to install ***Mocha***. Mocha is a testing framework that is used to perform tests within our application. It makes sure everything works correctly. 'Mocha in Node.js' means using the mocha unit testing framework in Node.js runtime environment. Mocha has tons of great features.

Key Features:

- Simple async support.
- Async test timeout support.
- Use any assertion library you want.

before, after, beforeEach, afterEach hooks are very useful to clean the environment with each test.

The **nodemon** Module is a module that develop node.js based applications by automatically restarting the node application when file changes in the directory are detected. **Nodemon** does not require any change in the original code and method of development.

Advantages of Using nodemon Module:

1. It is easy to use and easy to get started.
2. It does not affect the original code and no instance require to call it.
3. It helps to reduce the time of typing the default syntax node <file name> for execution again and again.

Mongoose.module is one of the most powerful external module of the node.js. Mongoose is a MongoDB ODM (Object database Modelling) that is used to translate the code and its representation from MongoDB to the Node.js server.

Advantages of Mongoose module:

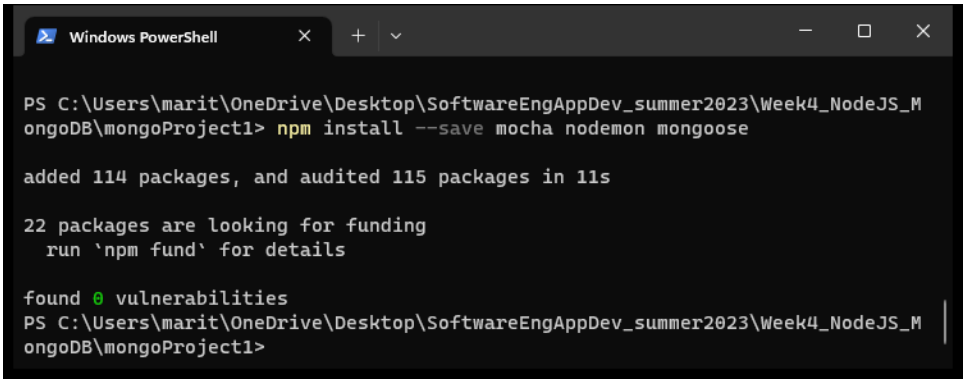
1. Collection validation of the MongoDB database can be done easily.
2. Predefined Structure can be implemented on the collection.
3. Constraints can be applied to documents of collections using Mongoose.

Mongoose module provides several functions in order to manipulate the documents of the collection of the MongoDB database. (Shriyan, 2021) (zack_aayush, 2020)

Let's install these two components from the node package manager. To install them, type

```
npm install --save mocha nodemon mongoose
```

in the Terminal

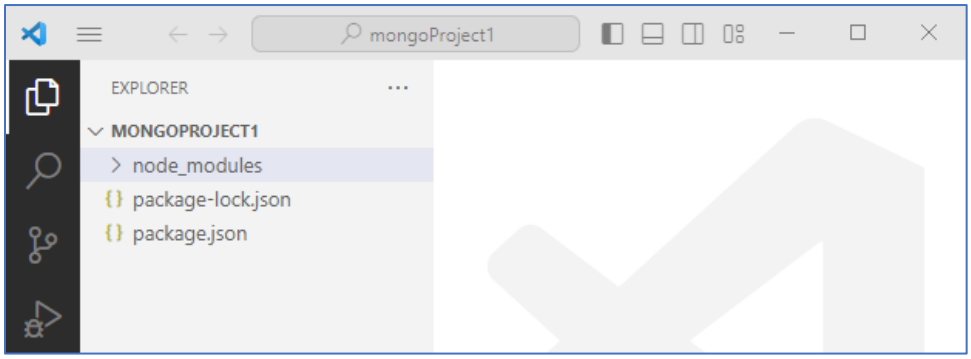


This is how we need the software to:

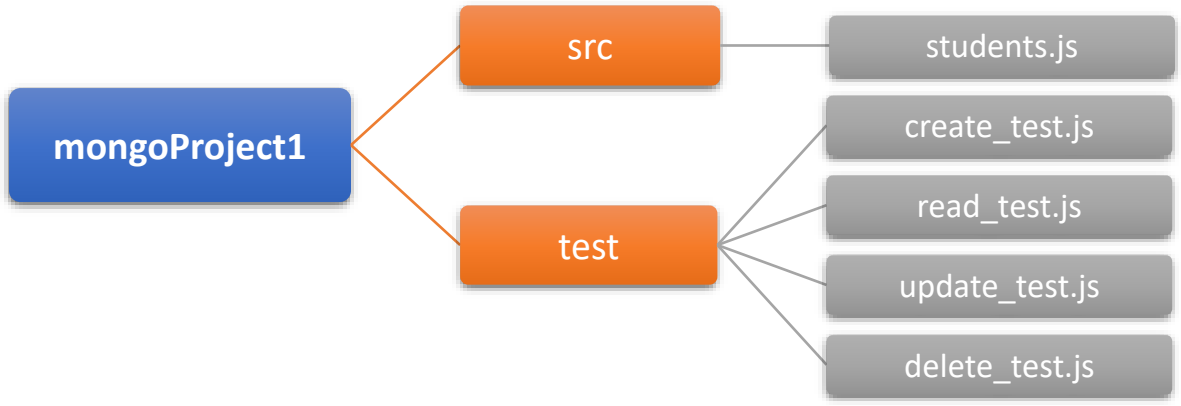
- We are doing to use ***mongoose*** to modify our database and then use ***mocha*** to check that we did alright with the modification in our database:



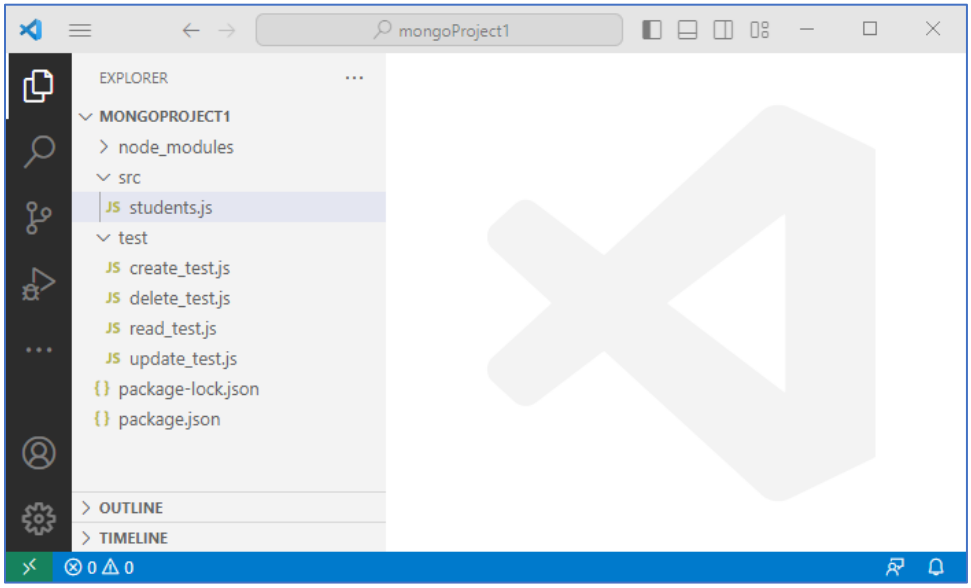
Now, let's open VS code text editor and open the project folder *mongoProject1*



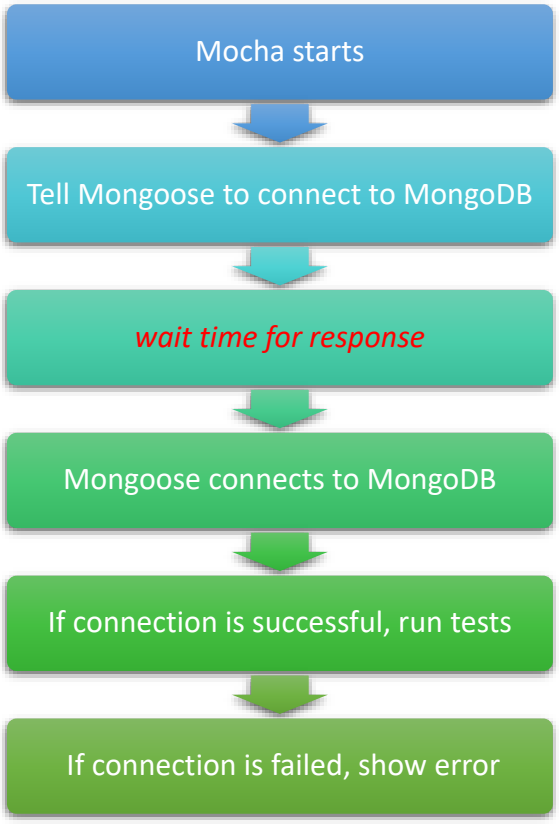
For this project, we are going to create the following structure of our files:



After creating the folders and files, in the VS code explorer should look as:



We also going to create a ***test_helper.js*** file within the ***test*** folder. This is used to test how the connection of the .js files with the MongoDB.



test_helper.js

```
// import mongoose
const mongoose = require('mongoose')

//connect to database, students_test is the db name.
mongoose.connect("mongodb://localhost/students_test")

//check if it is connected
mongoose.connection
    //.once watches for mongoDB to connect the first time the event occurred
    .once('open', () => { console.log('\n----- Connected to mongoDB ----- \n') })
    // .on watches for error in the connection
    .on('error', (error) => {
        console.log('Error connecting ', error)
    })
```

Let's run the file and see how the *test_helper.js* is doing. Open the terminal in VS code and type the path to the folder:

```
node test/test_helper
```

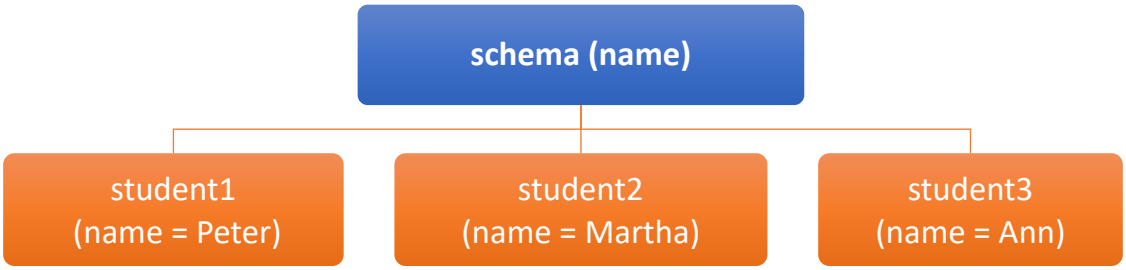
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL                                    node

PS C:\Users\marit\OneDrive\Desktop\SoftwareEngAppDev_summer2023\Week4_NodeJS_MongoDB\mong
oProject1> node test/test_helper.js

----- Connected to mongoDB -----
```

Use Mongoose to create a model

To create a model, we first design the model schema to understand how the model will look like. For our project, we are going to create the following model schema for students.



A database schema is a **blueprint that outlines how data is organized and stored**. The permitted Schema types in mongoose are:

- String
- Number
- Date
- Buffer
- Boolean
- Mixed
- Objected
- Array
- Decimal128
- Map

This student model schema will be written in the *students.js* file:

```
JS students.js ×
src > JS students.js > ...
1   //import mongoose library
2   const mongoose = require('mongoose')
3   const Schema = mongoose.Schema
4
5   //config the schema
6   const StudentSchema = new Schema({
7       name: String
8   })
9
10  //create the model schema for each student
11  const Student = mongoose.model('student', StudentSchema)
12
13  module.exports = Student;
```

Running test with mocha

Having the model, the next step is to create a data for one student and then test if the program is working properly. The program should start to create a student data and transfer it to the database. To create the student data, we open ***create_test.js*** file:

```
JS create_test.js ×
test > JS create_test.js > ...
1   // import assert . assert is used to compare two Boolean values
2   const assert = require('assert')
3
4   // create describe function
5   describe('Create the first data' ,()=>{
6       it('Save the student', ()=>{
7           assert((2+8) ===10)
8       })
9   })
```

it functions pushed data to a list that will be executed either by the **describe** function or by mocha itself. The it functions do NOT execute the tests.

In order to test if the file is working, we need to set up the command in the *package.json* file and change the *scripts* key value to *mocha*

create_test.js package.json

package.json > scripts

```
{
  "name": "mongoproject1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  Debug
  "scripts": {
    "test": "mocha"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "mocha": "^10.2.0",
    "mongoose": "^7.4.4",
    "nodemon": "^3.0.1"
  }
}
```

We can run a test by typing ***npm run test***

create_test.js

test > create_test.js > ...

```
// import assert . assert is used to compare two Boolean values
const assert = require('assert')

// create describe function
describe('Create the first data' ,()=>{
    it('Save the student', ()=>{
        assert((2+8) ===10)
    })
})
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

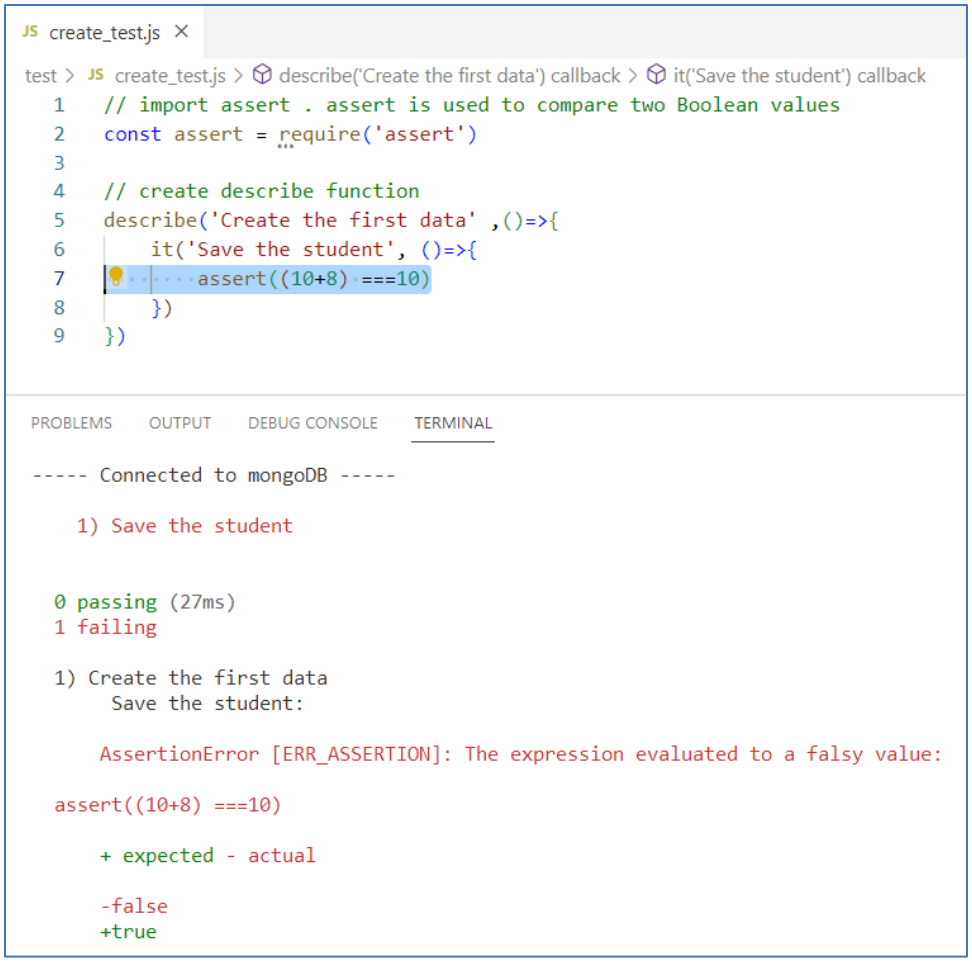
```
PS C:\Users\marit\OneDrive\Desktop\SoftwareEngAppDev_summer2023\Week4_NodeJS_MongoDB\mongoProject1> npm run test

> mongoproject1@1.0.0 test
> mocha


  Create the first data
    ✓ Save the student


----- Connected to mongoDB -----


  1 passing (19ms)
```



Now, let's change the *assert()* to be false and run the test again, mocha check will show an error:

In the terminal we can see that the *describe()* method was properly run. So the main aim here is to write some tests to make sure that the information is passing through mocha. Once the test is successful run, then we are able to create a new student and save the database.

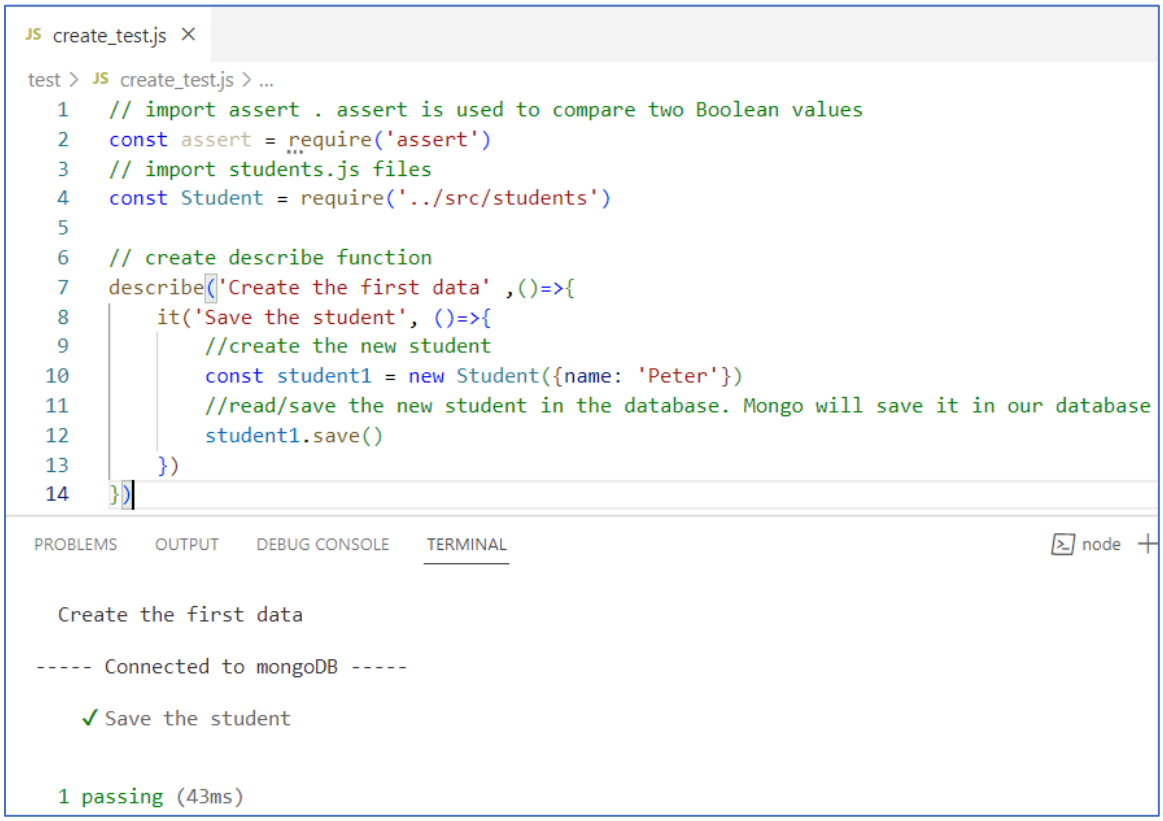
Creating new variables

After the test is completed, the next step is to import the students.js file to work with the student model:

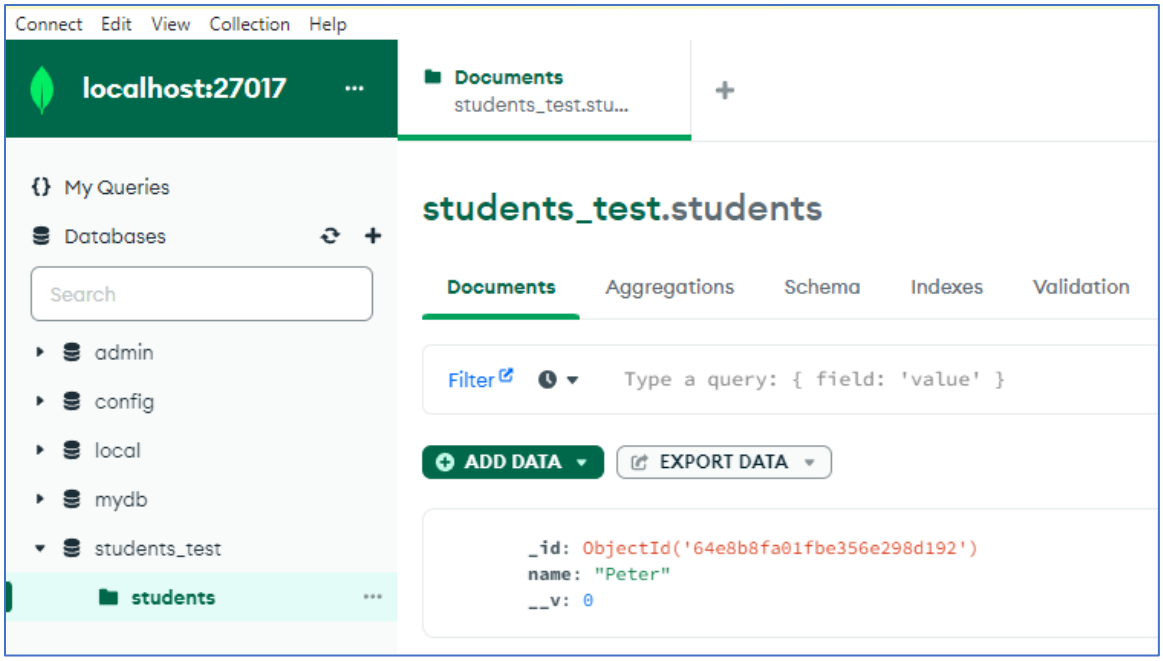
```
const Student = require('../src/students')
```

We capitalized the letter ***S*** to indicate that is the special class of student.

We create the new student inside the ***it*** statement using the constructor Student and then save the new student to the database using the mocha ***save()*** method:

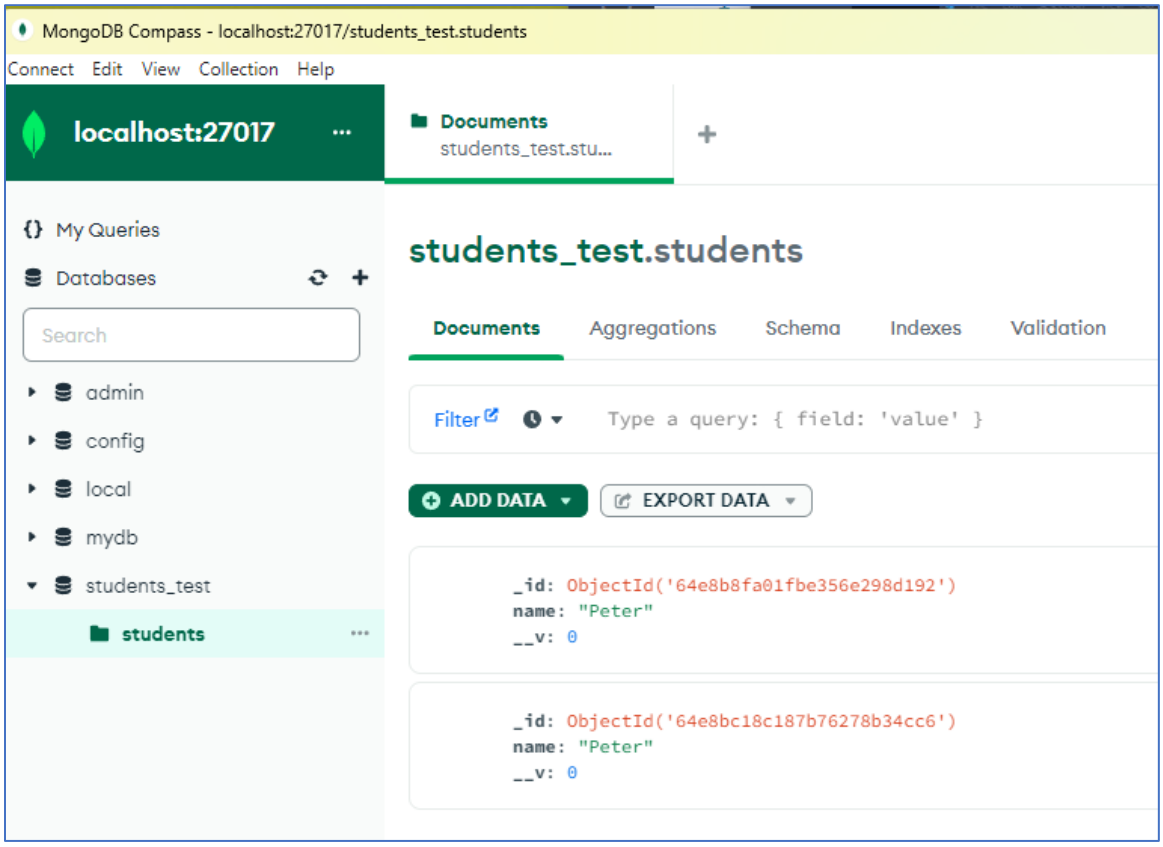


If we see a green check mark, it means that the information is successfully transferred to our database. To check that information, we can open ***compass*** and connect to our database:

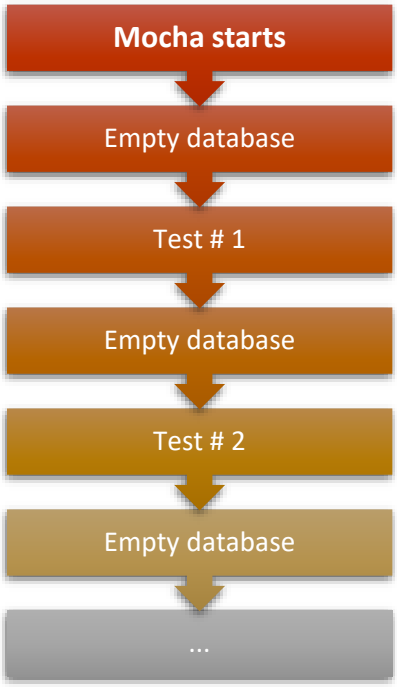


Dropping data from a database

Every time we run the *create_test.js* file, a new data will be sent to the database:



To avoid those inconsistencies, we can clean up every time that we run our test in the test suite. To do so, we can write the code according to the following schema:



This cleaning process will make that every test we run, it is in isolation and there will not be any remainder of another test.

To do so, we are going to use the ***beforeEach()*** function in the *test_helper.js* file. *beforeEach()* is a hook and it will run before each test, so it'll empty out our database and then run our test. To clean all the records before the running of each test, use the *correction.collections* methods of mongoose:

```
mongoose.connection.collections.students.drop()
```

We write the *drop()* method at the end to delete all the records inside of the students collection.

For the *beforeEach()* function, we pass a parameter ***done*** to make *beforeEach()* a callback function and it will be executed when all the students are done:

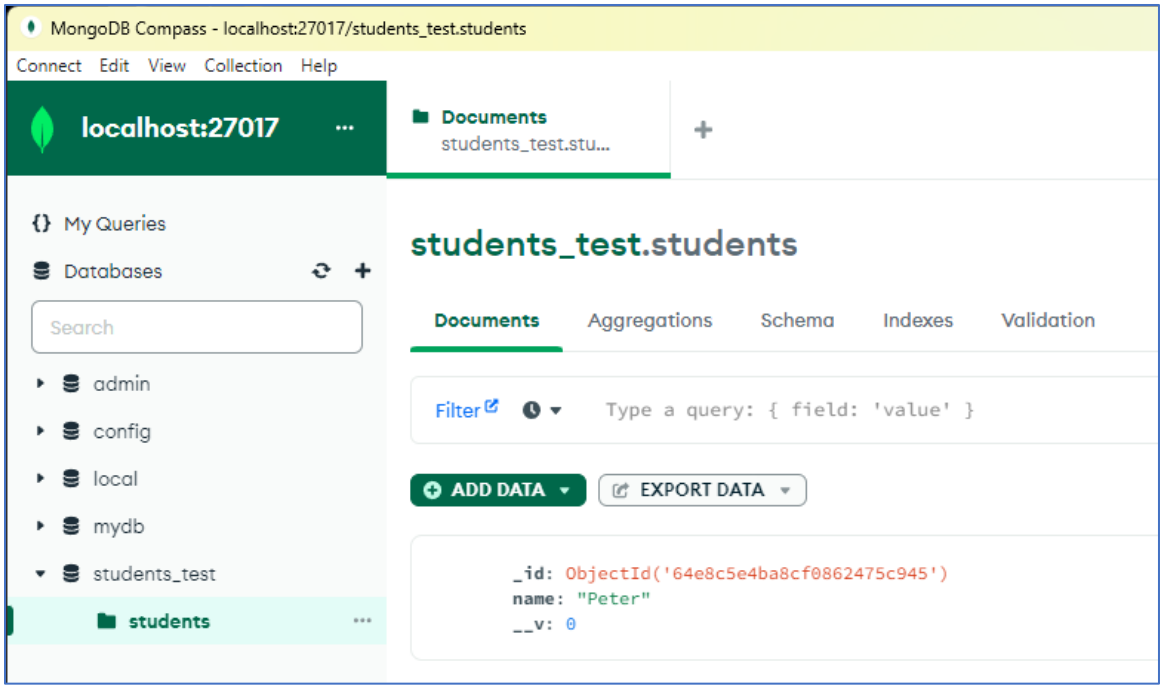
```
beforeEach((done)=>{
    mongoose.connection.collections.students.drop()
```

```
        done()
    })
```

We write *done* as an argument in *beforeEach()* to tell Mocha that if I'm calling this function because I am completing the function and it is fine to run the next one or next call.



Writing the app in the terminal, our database gets updated to and only one record is saved:



Insert data in a database = promises

The Node.js driver uses the asynchronous Javascript API to communicate with your MongoDB cluster.

Asynchronous Javascript allows you to execute operations without waiting for the processing thread to become free. This helps prevent your application from becoming unresponsive when executing long-running operations.

This section describes `Promises` that you can use with the Node.js driver to access the results of your method calls to your MongoDB cluster.

A Promise is an object returned by the asynchronous method call that allows you to access information on the eventual success or failure of the operation that they wrap. The Promise is in the **Pending** state if the operation is still running, **Fulfilled** if the operation completed successfully, and **Rejected** if the operation threw an exception.

You can define your own logic that executes once the Promise reaches the **Fulfilled** or **Rejected** state by appending the `then()` method. The first parameter of `then()` is the method that gets called when the Promise reaches the **Fulfilled** state

and the optional second parameter is the method that gets called when it reaches the **Rejected** state. The `then()` method returns a Promise to which you can append additional `then()` methods.

create_test.js U

test > create_test.js > ...

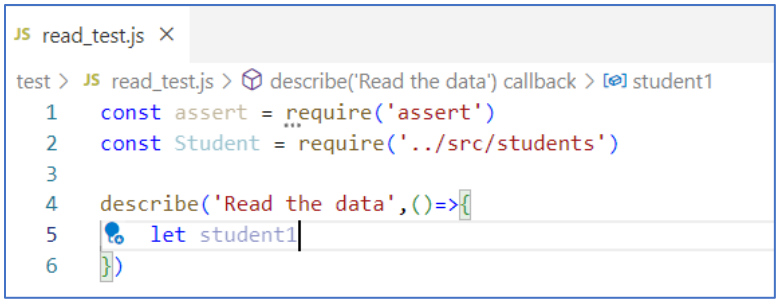
```
1  //import assert. assert is used to compare two Boolean values
2  const assert = require("assert")
3  // import student's schema
4  const Student = require('../src/students')
5
6  // describe function to push testing through Mocha
7  describe('Create the first data', function(){
8      it('Saved the student', function(){
9          //create a new student
10         const student1 = new Student({name:"Peter"})
11
12         //read and save the student in the database. Mongo will save the new student in the
           database
13         student1.save()
14
15         // check if the student is not new using the .then method.
16         // if the student is new, the promise reaches the 'fulfilled' state and will proceed to
           create the schema for the new student. Otherwise, the promise reaches the 'rejected'
           state and will proceed to reject line 'student1.save()'
17         .then(function(){
18             assert(!student1.isNew)
19         })
20
21     })
22 })
```

Reading data

Find() query

So far, we created an instance of a new student, then saved it and asserted that the new student was successfully saved through the database. Now it is time to find the records that we have created in our database. They are different ways to find out if a record is in or out of the database. We can find data using a unique id number or using the name. Since the name might not be unique, then we can find the student by the id number.

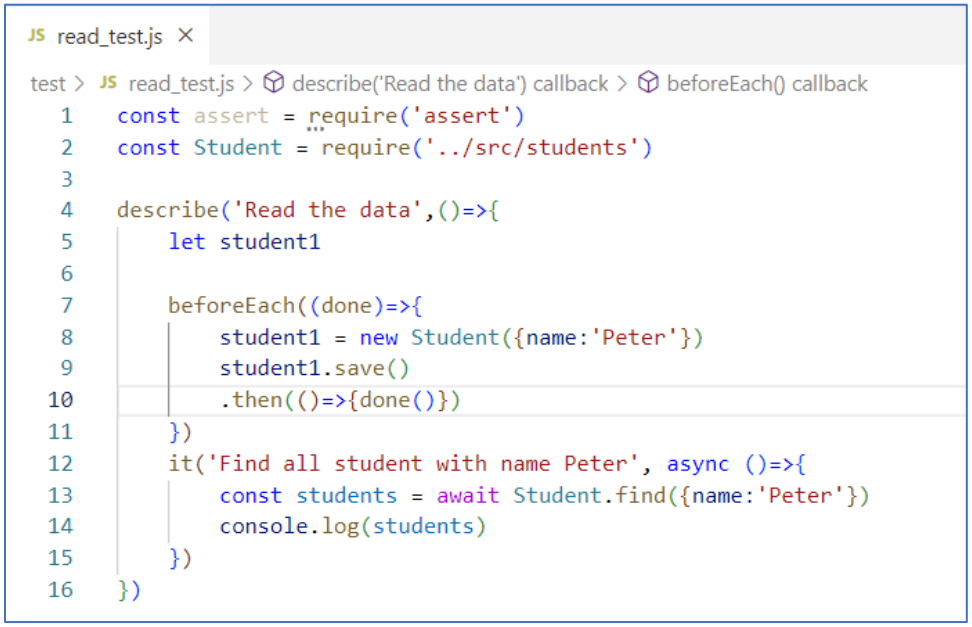
To do so, under the *test* directory, we are going to open the *read_test.js* and prepare our data:



They are two different query methods and mongoose library to find data:

- Student.find(criteria) → returns an array.
- Student.findOne(*criteria)* → returns a single record.

Criteria is the property that we are looking for, in our case, we are looking for a student with name Peter:



Running the Terminal:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL                    node

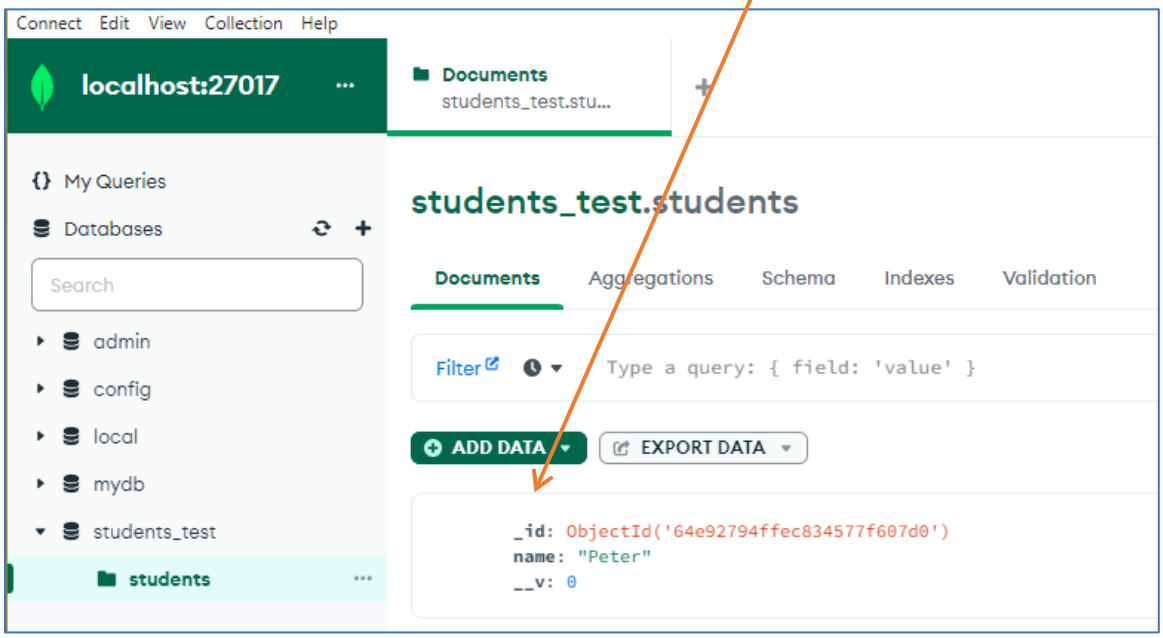
  Create the first data

----- Connected to mongoDB -----

    ✓ Save the student (43ms)

  Read the data
[
  {
    _id: new ObjectId("64e8d055283ee08969900afa"),
    name: 'Peter',
    __v: 0
  }
]
    ✓ Find all student with name Peter
```

Now, to check or make sure that the value that we are checking is the one we are looking for. For this case, we can check for the proper value by using the MongoDB **_id**:



We are going to test if *student1* id is the same as the one saved in the database:



Once *student1* id is checked, now we can use assert that the first student in array *students* has the same id as the one we are looking for:

```
assert(students[0]._id.toString()===student1._id.toString()
)
```

The complete *read_test.js* file looks as follow:

read_test.js

test > read_test.js > describe('Read the data') callback > it('Find all student with name Peter') callback

```
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4   describe('Read the data',()=>{
5       let student1
6
7       beforeEach((done)=>{
8           student1 = new Student({name:'Peter'})
9           student1.save()
10          .then(()=>{done()})
11      })
12      it('Find all student with name Peter', async ()=>{
13          const students = await Student.find({name:'Peter'})
14          console.log(students)
15          assert(students[0]._id.toString()===student1._id.toString())
16      })
17  })
```

Automatic testing

We can run automatic testing by modifying the value of *scripts* to `nodemon --exec mocha -R min`

in the ***package.json***:

package.json

{} package.json > {} scripts > test

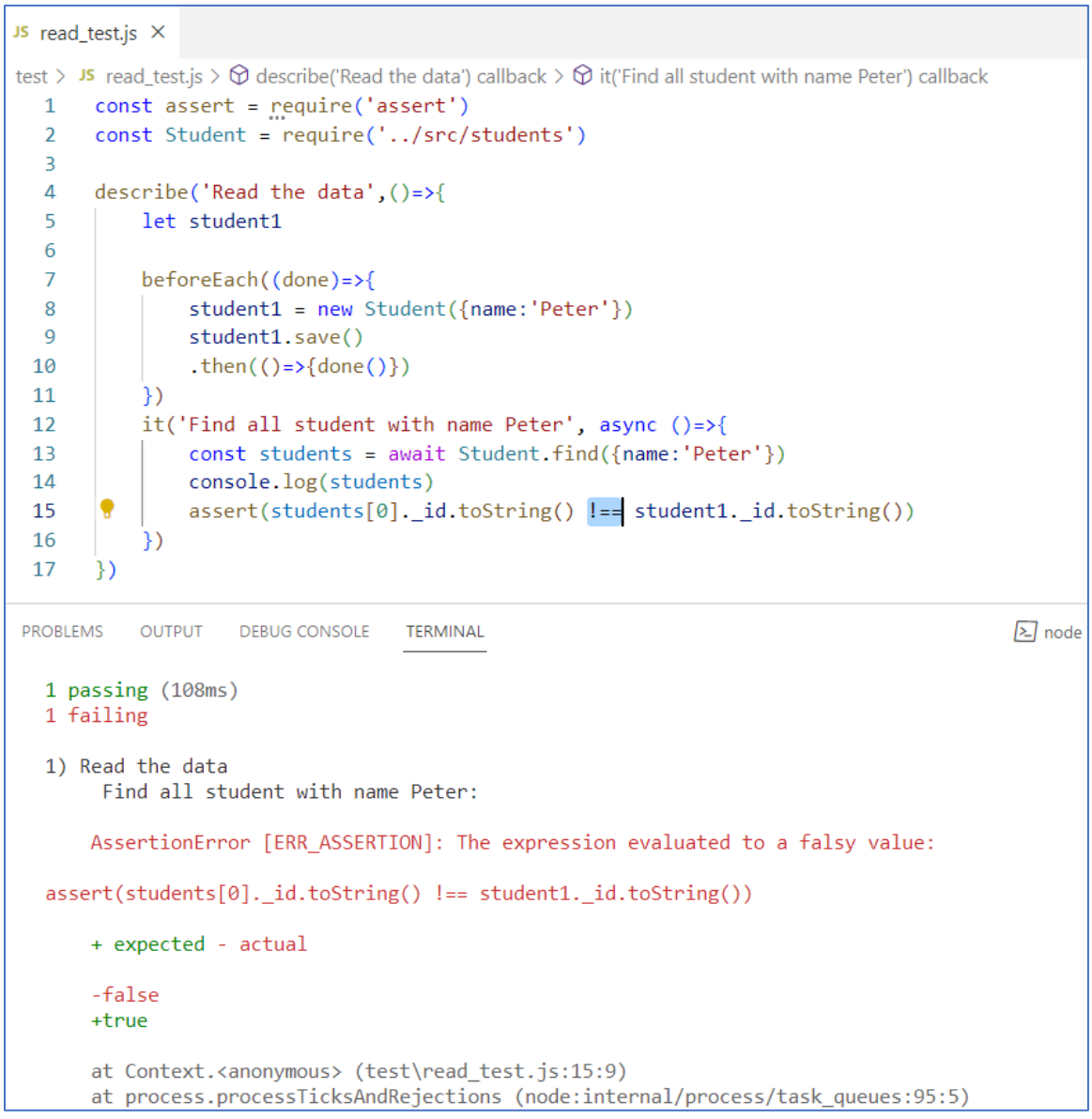
```
1   {
2     "name": "mongoproject1",
3     "version": "1.0.0",
4     "description": "",
5     "main": "index.js",
      Debug
6     "scripts": {
7       "test": "nodemon --exec mocha -R min"
8     },
9     "author": "",
10    "license": "ISC",
11    "dependencies": {
12      "mocha": "^10.2.0",
13      "mongoose": "^7.4.4",
14      "nodemon": "^3.0.1"
15    }
16  }
```

`-R min` are about the formatting of the test output and it clears the outputs remaining for many of the previous runs.

Now, we can run the test in the Terminal, ***npm run test***. The Terminal runs two test at the time. We can modify to code for error in the *read_test.js*

```
    assert(students[0]._id.toString() !== student1._id.toString())
```

and we can see the testing result minimized:



findOne() query



Deleting data

There are different methods to delete data from database. For this example, we are going to try different methods. Let's setting up our *delete_test.js* file:

delete_test.js | package.json

test > delete_test.js > describe('Delete the records') callback > it('delete

```
1  const assert = require('assert')
2  const Student = require('../src/students')
3
4  describe('Delete the records', ()=>{
5      let student1
6
7      beforeEach((done)=>{
8          student1 = new Student({name: 'Peter'})
9          student1.save()
10         .then(()=> done())
11     })
12
13     it('delete by id', done=>{
14         Student.findByIdAndDelete(student1._id)
15         .then(()=>Student.findOne({name:'Peter'}))
16         .then((student) =>{
17             assert(student === null)
18             done()
19         })
20     })
```

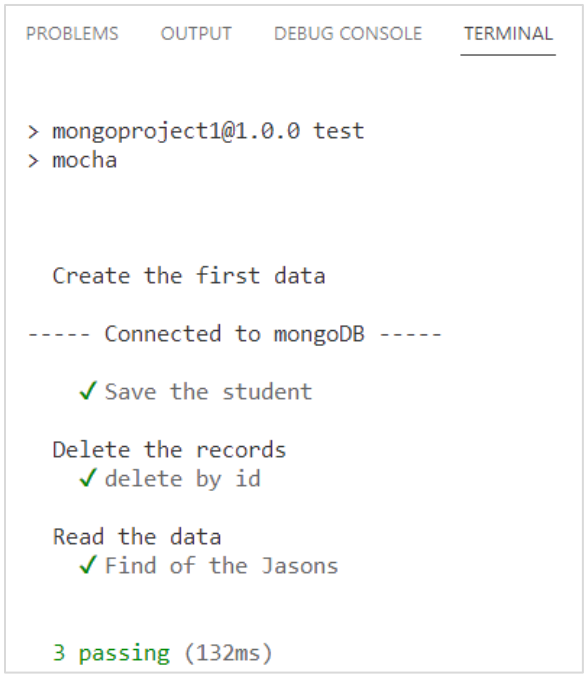
Before running the test to the program, we can change the test method back to *mocha*

package.json

{} package.json > {} scripts

```
1  {
2    "name": "mongoproject1",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
       Debug
6    "scripts": {
7      "test": "mocha"
8    },
9    "author": "",
10   "license": "ISC",
11   "dependencies": {
12     "mocha": "^10.2.0",
13     "mongoose": "^7.4.4",
14     "nodemon": "^3.0.1"
15   }
16 }
```

Running the test in the terminal:



Now we can also delete by name using the method *findOneAndDelete():*

```
it('delete by name', done=>{
    Student.findOneAndDelete({name: 'Peter'})
    .then(()=>Student.findOne({_id : student1._id}))
    .then((student) =>{
        assert(student === null)
        done()
    })
})
```

Running the test in the Terminal:

```
  Create the first data

----- Connected to mongoDB -----

    ✓ Save the student (39ms)

  Delete the records
    ✓ delete by id
    ✓ delete by name

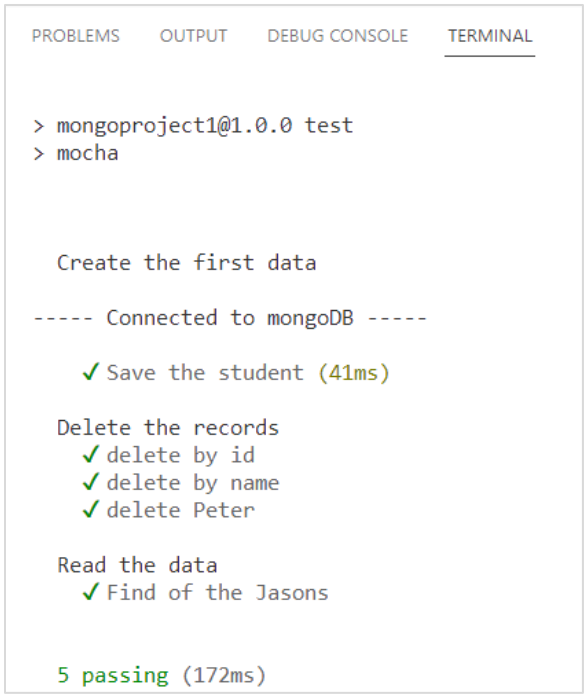
  Read the data
    ✓ Find of the Jasons


  4 passing (144ms)
```

Now let's try the method *deleteOne()* :

```
it('delete Peter', done=>{
    Student.deleteOne({_id:student1._id})
    .then(()=>Student.findOne({name:'Peter'}))
    .then((student) =>{
        assert(student === null)
        done()
    })
})
```

Running the test:



Remember, all these are not happening in the database, only happen in the memory.

Update the database

After the delete, we need to update the data in the database. There are two types of methods to update the database:

- Model class: `updateOne()`, and `updateMany()`
- Model instance: `set` and `save`

Set and save method

Let's open the *update_test.js* file and prepare the file:

update_test.js

test > update_test.js > describe('Updating records') callback > it('Set and save') callback

```
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4   describe('Updating records',()=>{
5       let student1
6
7       beforeEach((done) =>{
8           student1 = new Student({name: 'Peter'})
9           student1.save()
10          .then(()=>done())
11      })
12
13      it('Set and save', done=>{
14          console.log(student1)
15          done()
16      })
17
18  })
```

We can run the testing:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
----- Connected to mongoDB -----

    ✓ Save the student

  Delete the records
    ✓ delete by id
    ✓ delete by name
    ✓ delete Peter

  Read the data
    ✓ Find of the Jasons

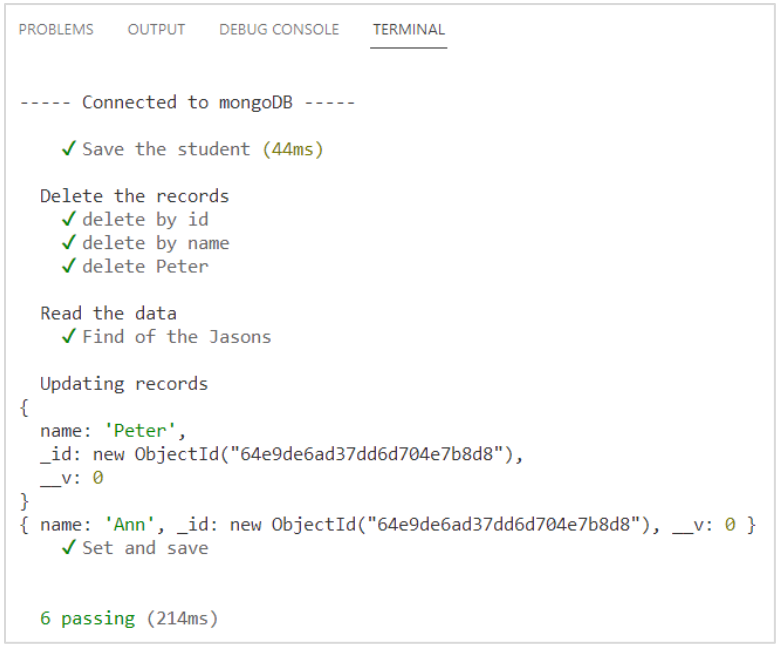
  Updating records
{
  name: 'Peter',
  _id: new ObjectId("64e9dd8e18db99a3a2d2fc73"),
  __v: 0
}
    ✓ Set and save


  6 passing (189ms)
```

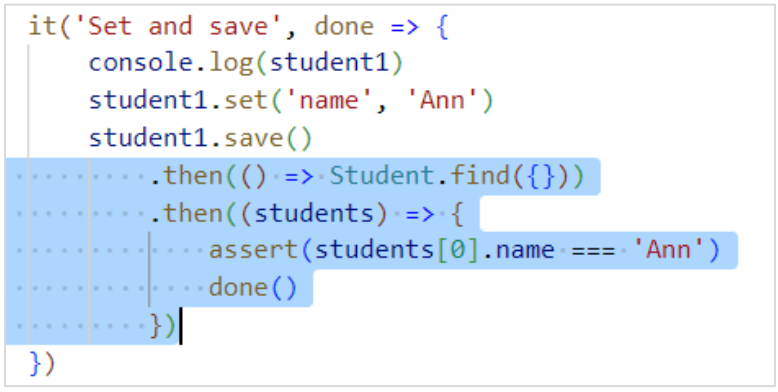
Let us set the name of *student1* to *Ann*:

```
it('Set and save', done=>{
    console.log(student1)
    student1.set('name','Ann')
    console.log(student1)
    done()
})
```

Running the terminal:



After the testing, we need to send the updates to the database:



Running the test:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

> mongoproject1@1.0.0 test
> mocha



  Create the first data

----- Connected to mongoDB -----

    ✓ Save the student (38ms)

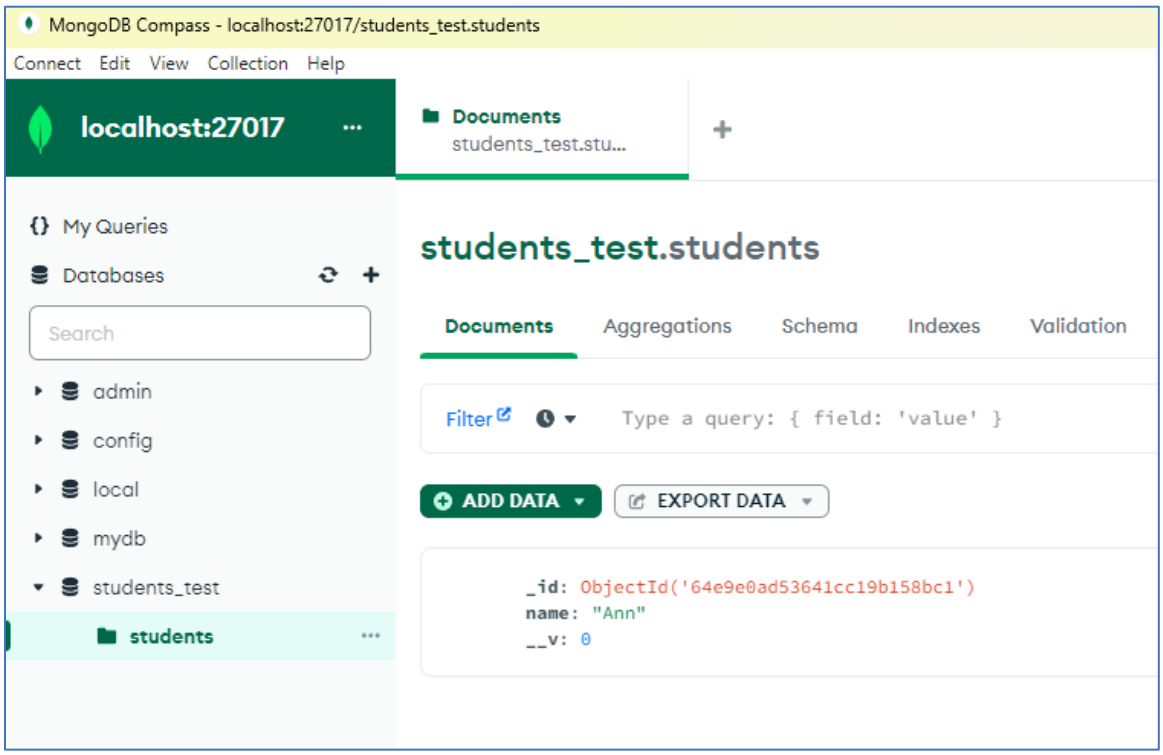
  Delete the records
    ✓ delete by id
    ✓ delete by name
    ✓ delete Peter

  Read the data
    ✓ Find of the Jasons

  Updating records
{
  name: 'Peter',
  _id: new ObjectId("64e9e0ad53641cc19b158bc1"),
  __v: 0
}
    ✓ Set and save


  6 passing (213ms)
```

The database will look as:



Update method

For this practice, let's up add one more schema as *studentNumber* in the *students.js* file:

students.js

```
src > students.js > StudentSchema
1   //import mongoose library
2   const mongoose = require('mongoose')
3   const Schema = mongoose.Schema
4
5   //config the schema
6   const StudentSchema = new Schema({
7       name: String,
8       studentNumber:Number
9   })
10
11  //create the model schema for each student
12  const Student = mongoose.model('student', StudentSchema)
13
14  module.exports = Student;
```

Now, let us go back to the *update_test.js* file and use the *upadate* method to update the student number:

update_test.js

```
test > update_test.js > ...
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4   describe('Updating records', () => {
5       let student1
6       let student2
7
8       beforeEach((done) => {
9           student1 = new Student({ name: 'Peter', studentNumber: 123 })
10          student2 = new Student({name:'Peter', studentNumber:210})
11          student2.save()
12          student1.save()
13              .then(() => done())
14          console.log(student1)
15          console.log(student2)
16      })
17
18      it('Update one of the Peters',async()=>{
19          const student = await Student.updateOne({name:'Peter'}, {studentNumber: 110})
20          const res = await Student.find({})
21          assert(res[0].studentNumber===110)
22          console.log(res)
23      })
24  })
```

Running the test:



We can also update many data or all the data at the time:

update_test.js

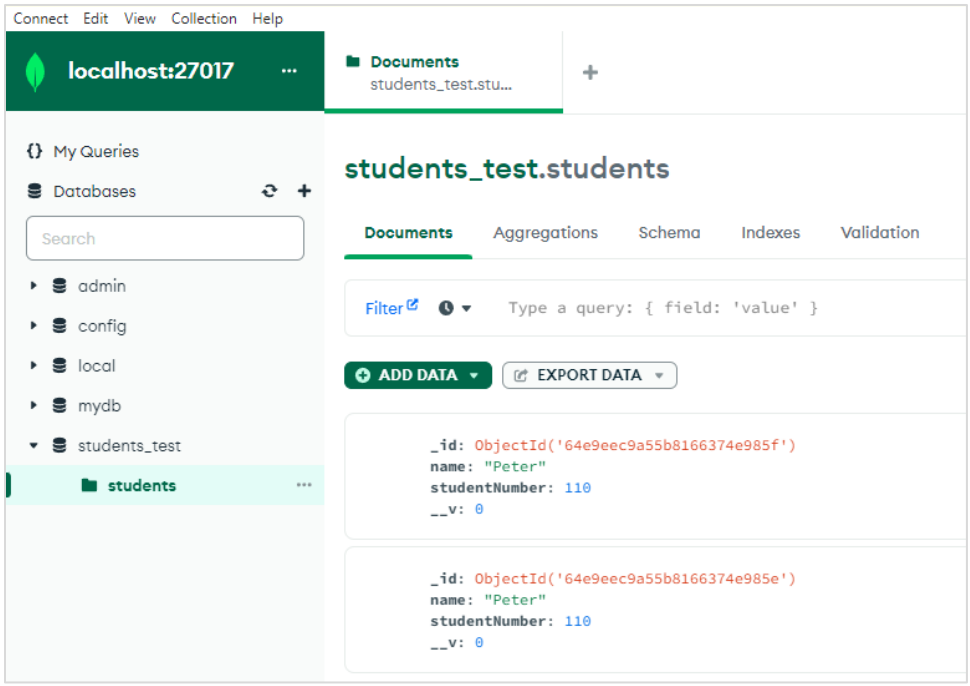
test > update_test.js > ...

```
4   describe('Updating records', () => {
5       let student1
6       let student2
7
8       beforeEach((done) => {
9           student1 = new Student({ name: 'Peter', studentNumber: 123 })
10          student2 = new Student({name:'Peter', studentNumber:210})
11          student2.save()
12          student1.save()
13              .then(() => done())
14          console.log(student1)
15          console.log(student2)
16      })
17      // change all student with name Peter to 110
18      it('Update all Peters',async()=>{
19          const student = await Student.updateMany({name:'Peter'}, {studentNumber: 110})
20          const res = await Student.find({})
21          assert(res[0].studentNumber===110 && res[1].studentNumber===110)
22          console.log(res)
23      })
24  })
```

Running the test:



Refreshing the database:



7.3.6. MongoDB Operator Types

There are nine types of operators, each named for its function. For instance, logical operators use logic operations. To execute them, you need to use a specific keyword and follow the syntax. However, they're fairly easy to follow!

By the end of the article, you'll be able to learn the basics of each operator and its functions.

Logical Operators

Logical operators are often used to filter data based on the given conditions. They also allow for the evaluation of many conditions, which we'll discuss in more detail.

Below are a few logical operators that you can use:

$and

An "and" condition performs a logical "and" operation on an array of two or more expressions. It selects the documents where *all* the conditions of the expressions are satisfied.

This is the standard syntax for the $and expression:

Syntax:

```
{ $and: [ { <expression1> }, { <expression2> }, ... , { <expressionN> } ] }
```

For example, if we want to select documents where the price is $10 and the quantity is less than 15, we can input the following query:

```
db.inventory.find( { $and: [ { quantity: { $lt: 15 } }, { price: 10 } ] } )
```

$or

An “or” condition performs a logical “or” operation on an array of two or more expressions. It selects the documents where at least one of the expressions is true.

This is the standard syntax for the $or expression:

Syntax:

```
{ $or: [ { <expression1> }, { <expression2> }, ... , { <expressionN> } ] }
```

For example, if we want to select documents where the price is $10 or the quantity is less than 15, we can input the following query:

```
db.inventory.find( { $or: [ { quantity: { $lt: 15 } }, { price:
10 } ] } )
```

We don't have to limit the expression to two criteria — we can add more. For instance, the below query selects those documents where the price equals $10, the quantity is under 15, or the tag is stationary:

```
db.inventory.find( { $or: [ { quantity: { $lt: 15 } }, { price:
10 }, { tag: "stationary" }] } )
```

When running these clauses, MongoDB either performs a collection scan or an index scan. If all indexes support the clauses, then MongoDB uses indexes to check an $or expression. Otherwise, it uses a collection scan instead.

But if you want to test the criteria in the same field, you may want to use the $in operator rather than the $or operator. For example, if you want a collection of documents where the quantity is either 10 or 20, you may have to run the below $in query instead:

```
db.inventory.find ( { quantity: { $in: [20, 50] } } )
```

$nor

This operator performs a logical "nor" operation on an array using one or more expressions. Next, it selects the documents that fail the query expressions. In simpler terms, it does the opposite of the $or condition.

This is the general syntax:

```
{ $nor: [ { <expression1> }, { <expression2> }, ...  { <expressionN> } ] }
```

Let's consider the following query:

```
db.inventory.find( { $nor: [ { price: 3.99 }, { sale: true } ]  } )
```

This query selects the documents that contain:

- a price field value not equal to $3.99, and a sale value not equal to true; or
- a price field value not equal to $3.99, and an empty or absent sale field; or
- no price field, and a sale field not equal to true; or
- neither price field nor sale field populated or present.

$not

This operator performs a logical "not" operation on an array for the specified expression. It then selects the documents that don't match the query expressions. This includes the documents that don't contain the field.

This is the general syntax:

```
{ field: { $not: { <operator-expression> } } }
```

For instance, take the following query:

```
db.inventory.find( { price: { $not: { $lt: 3.99 } } } )
```

This query would select those documents that contain:

- a price field whose value is greater than or equal to $3.99; and
- a price field is unpopulated or does not exist.

Info

`{ $not: { $lt: 3.99 } }` is different from the `$gte` operator. `{ $gte: 3.99 }` returns only the documents where the price field exists and its value is greater than or equal to $3.99 (the `$not` operator returns even those documents where the price field doesn't exist).

(Ravoof, 2023)

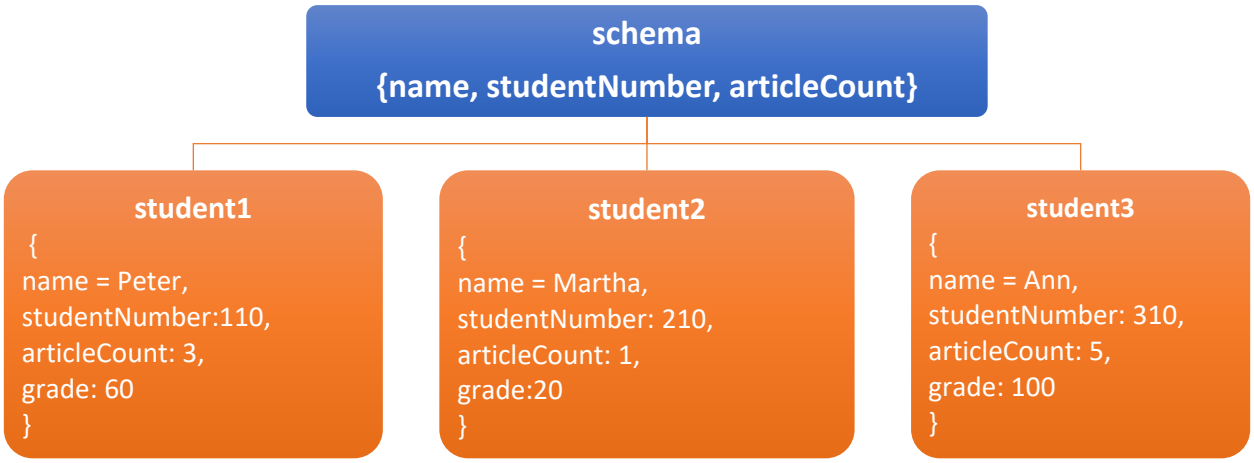
MongoDB provides different types of field update operators to update the values of the fields of the documents that matches the specified condition. The following table contains the field update operators:

Operator	Description
$currentDate	This operator is used to set the value of a field to current date, either as a Date or a Timestamp.
$inc	This operator is used to increment the value of the field by the specified amount.
$min	This operator is used only to update the field if the specified value is less than the existing field value
$max	This operator is used only to update the field if the specified value is greater than the existing field value.
$mul	This operator is used to multiply the value of the field by the specified amount.

Operator	Description
$rename	This operator is used to rename a field.
$setOnInsert	This operator is used to set the value of a field if an update results in an insert of a document. It has no effect on update operations that modify existing documents.

(Ankita_Saini, MongoDB – Field Update Operators, 2023)

For our example, we are going to add another document, *articleCount* and *grade,* to our *student* schema. *articleCount* will be used as the number of articles posted by each student and *grade* will be student's grade for their articles. The new schema will look as:



Open *students.js* file and at document *articleCount* and *grade:*

students.js

src > students.js > StudentSchema

```
1   //import mongoose library
2   const mongoose = require('mongoose')
3   const Schema = mongoose.Schema
4
5   //config the schema
6   const StudentSchema = new Schema({
7       name: String,
8       studentNumber:Number,
9       articleCount: Number,
10      grade: Number
11  })
12
13  //create the model schema for each student
14  const Student = mongoose.model('student', StudentSchema)
15
16  module.exports = Student;
```

Now, open *update_test.js* file and update the information in *beforeEach():*

update_test.js

test > update_test.js > ...

```
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4   describe('Updating records', () => {
5       let student1, student2, student3
6
7       beforeEach((done) => {
8           student1 = new Student({ name: 'Peter', studentNumber: 110, articleCount:3, grade:60 })
9           student2 = new Student({name:'Martha', studentNumber:210, articleCount:1, grade:20})
10          student3 = new Student({name:'Ann',studentNumber:310, articleCount:5, grade:100 })
11          student3.save()
12          student2.save()
13          student1.save()
14              .then(() => done())
15          console.log(student1)
16          console.log(student2)
17          console.log(student3)
18      })
19  })
```

Now the information is static, However, I also want to increase this grade, according to a number of articles that the students have. We cannot do this just to update operators instead. There are mongo operators to do this ____

The data transfer works as:

- MongoDB fetches every student,
- load students into the server,
- update the grids and multiple with the article numbers
- save the wreckage that base again.

This process works but it will take a huge performance hit.

To allow the performance hit with larger amounts of data, we can use the test suit to send a message or instructions to our database to find the students that we're looking for. To do this, we are going to use the multiply operator, $mul , from mongoDB update operators.

MongoDB provides different types of field update operators to update the values of the fields of the documents and $mul operator is one of them. This operator is used to multiply the value of the field by a number.

- $mul operator only updates those fields whose value are of numeric type like int, float, etc.
- If the specified field is not present in the document, then this operator will add that field in the document and assign the value of that field to zero of the same numeric type as the multiplier.
- This operator is an atomic operation within a single document.

In this operator, multiplication with values of mixed numeric types like 32-bit integer, 64-bit integer, float, etc., may result in conversion of numeric type. The following rules are applied in the multiplication with the values of mixed numeric types:

	32-bit Integer	64-bit Integer	Float
32-bit Integer	32-bit or 64-bit Integer	64-bit Integer	Float
64-bit Integer	64-bit Integer	64-bit Integer	Float
Float	Float	Float	Float

$mul operator can also work with embedded/nested documents or arrays. You can use this operator in methods like update(), findAndModify(), etc., according to your requirements.

Syntax:

```
{ $mul: { <field1>: <number1>, <field2>: <number2>, ... } }
```

Here, `<field>` can specify with dot notation in embedded/nested documents.

For our example, we are doing to update the grade to student according to their number of articles. Let's update try the update Martha's grade. We are going to change the articleCount to 2, and then update the grade by multiplying the current grade with the number of articles:

update_test.js

test > update_test.js > describe('Updating records') callback

```
const assert = require('assert')
const Student = require('../src/students')

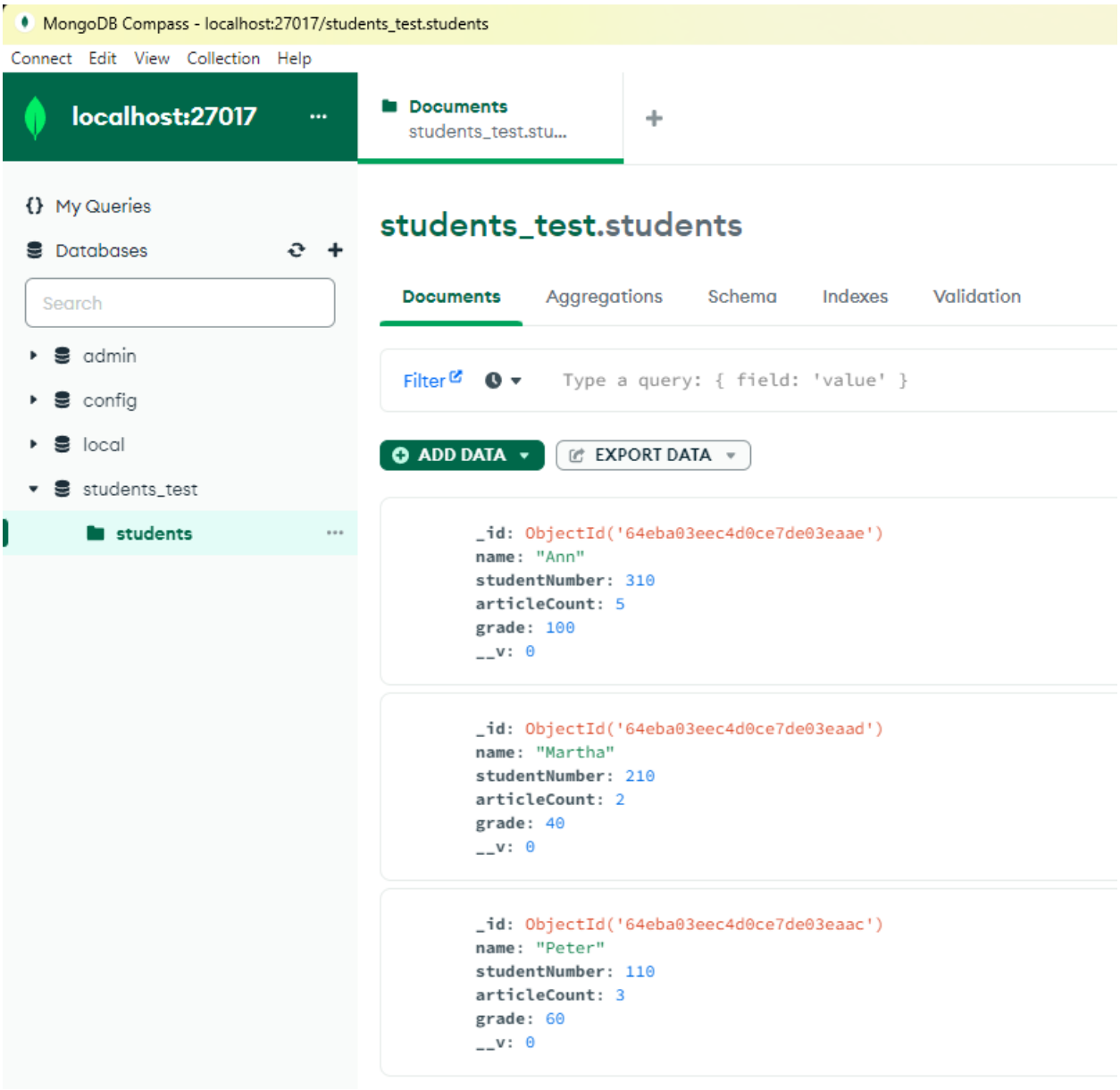
describe('Updating records', () => {
    let student1, student2, student3
    const gradeperArticle = 20

    beforeEach((done) => {
        student1 = new Student({ name: 'Peter', studentNumber: 110, articleCount: 3, grade: gradeperArticle })
        student2 = new Student({ name: 'Martha', studentNumber: 210, articleCount: 2, grade: gradeperArticle })
        student3 = new Student({ name: 'Ann', studentNumber: 310, articleCount: 5, grade: gradeperArticle })
        student3.save()
        student2.save()
        student1.save()
            .then(() => done())
    })

    it("update grades", async () => {
        const artCount = await Student.findOne({ name: 'Martha' })
        const student = await Student.updateOne({ name: 'Martha' }, { $mul: { grade: artCount.articleCount } })
        const res = await Student.find({ name: 'Martha' })
        assert(res[0].grade === (gradeperArticle * artCount.articleCount))
    })

})
```

Running the code and refreshing the database:


MongoDB Compass - localhost:27017/students_test.students
Connect Edit View Collection Help
localhost:27017
Documents
students_test.stu...
My Queries
Databases
Search
admin
config
local
students_test
students
students_test.students
Documents
Aggregations
Schema
Indexes
Validation
Filter
Type a query: { field: 'value' }
ADD DATA
EXPORT DATA
_id: ObjectId('64eba03eec4d0ce7de03eaae')
name: "Ann"
studentNumber: 310
articleCount: 5
grade: 100
__v: 0
_id: ObjectId('64eba03eec4d0ce7de03eaad')
name: "Martha"
studentNumber: 210
articleCount: 2
grade: 40
__v: 0
_id: ObjectId('64eba03eec4d0ce7de03eaac')
name: "Peter"
studentNumber: 110
articleCount: 3
grade: 60
__v: 0

7.3.7. Mongoose Validation

Mongoose Validation is essentially a customisable middleware that gets defined inside the SchemaType of mongoose schema. It automatically fires off before a document is saved in the noSQL DB. Validation can also be run manually using doc.validate(callback) or doc.validateSync() methods. Let's understand more about this with the help of some examples. (dishebhhayana, 2022)

For our example, let's create a validation to check if a student has a name that is longer than 2 characters.

We start by adding new properties to the object name in the file students.js

students.js

```
src > students.js > StudentSchema
1   //import mongoose library
2   const mongoose = require('mongoose')
3   const Schema = mongoose.Schema
4
5   //config the schema
6   const StudentSchema = new Schema({
7       name: {
8           type: String,
9           required : [true, 'Name is required']
10      },
11      studentNumber:Number,
12      articleCount: Number,
13      grade: Number
14  })
15
16  //create the model schema for each student
17  const Student = mongoose.model('student', StudentSchema)
18
19  module.exports = Student;
```

After it, we are creating a new file validation_test.js. In this file we are writing a code to validate if the length of the student's name is longer than 2 characters:

validation_test.js

```
test > validation_test.js > ...
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4
5   describe('validation', ()=>{
6       it('name is required',()=>{
7           const student = new Student({name: undefined})
8           const result = student.validateSync()
9           console.log(result)
10      })
11  })
```

Running the test in the terminal:

validation_test.js

```
test > validation_test.js > ...
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4
5   describe('validation', ()=>{
6       it('name is required',()=>{
7           const student = new Student({name: undefined})
8           const result = student.validateSync()
9           console.log(result)
10      })
11  })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
ongoDB\mongoProject1\node_modules\mocha\lib\runner.js:559:5)
        at process.processImmediate (node:internal/timers:478:21) {
      properties: [Object],
      kind: 'required',
      path: 'name',
      value: undefined,
      reason: undefined,
      [Symbol(mongoose:validatorError)]: true
    }
  },
  _message: 'student validation failed'
}
    ✓ name is required


  7 passing (244ms)
```

Now, in order to avoid unnecessary complications, we can write consed message within a curly brace equals to the resultant errors that name that message and that enables us to get the error message instead of a complete error message.

```
const{message} = result.errors.name
```

validation_test.js

```
test > validation_test.js > describe('validation') callback
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4
5   describe('validation', ()=>{
6       it('name is required',()=>{
7           const student = new Student({name: undefined})
8           const result = student.validateSync()
9           const{message} = result.errors.name
10          console.log(message)
11      })
12  })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
    ✓ delete by name
    ✓ delete Peter

  Read the data
    ✓ Find of the JPeters

  Updating records
    ✓ update grades

  validation
Name is required
    ✓ name is required


  7 passing (237ms)
```

After it, we can assert the validation:

validation_test.js

```
test > validation_test.js > describe('validation') callback > it('name is required') callback >
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4   describe('validation', ()=>{
5       it('name is required',()=>{
6           const student = new Student({name: undefined})
7           const result = student.validateSync()
8           console.log(result)
9           const{message} = result.errors.name
10          assert(message === 'Name is required')
11      })
12  })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
        at process.processImmediate (node:internal/timers:478:21) {
      properties: [Object],
      kind: 'required',
      path: 'name',
      value: undefined,
      reason: undefined,
      [Symbol(mongoose:validatorError)]: true
    }
  },
  _message: 'student validation failed'
}
    ✓ name is required


  7 passing (255ms)
```

required in mongoose is only validating if a name is provided or not. But we also want to check if the length of the name is longer than 2 characters. For this, we have to add another key, validator, in the students.js file to validate the length of the name. The validator key has two properties:

- first property is called validator and it will be called as a function with a name.
- second property is the message that we want to display if there is a fail and a validate function.

students.js

src > students.js > ...

```
1   //import mongoose library
2   const mongoose = require('mongoose')
3   const Schema = mongoose.Schema
4
5   //config the schema
6   const StudentSchema = new Schema({
7       name: {
8           type: String,
9           required : [true, 'Name is required'],
10          validate: {
11              validator: (name)=> name.length > 2,
12              message: 'Name is too short'
13          }
14      },
15      studentNumber:Number,
16      articleCount: Number,
17      grade: Number
18  })
19
20  //create the model schema for each student
21  const Student = mongoose.model('student', StudentSchema)
22
23  module.exports = Student;
```

In the file

validation_test.js

test > validation_test.js > ...

```
1   const assert = require('assert')
2   const Student = require('../src/students')
3
4   describe('validation', ()=>{
5       it('name is required',()=>{
6           const student = new Student({name: undefined})
7           const result = student.validateSync()
8           const{message} = result.errors.name
9           assert(message === 'Name is required')
10      })
11
12      it('Name must be longer than 2 characters', ()=>{
13          const newStudent = new Student({name:'A'})
14          const newResult = newStudent.validateSync()
15          const{newMessage} = newResult.errors.name
16          assert(newMessage === 'Name is too short')
17      })
18  })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
Name must be longer than 2 characters:

AssertionError [ERR_ASSERTION]: The expression evaluated to a falsy value:

  assert(newMessage === 'Name is too short')

    + expected - actual

    -false
    +true
```

References

Ankita_Saini. (2021, June). *What is MongoDB – Working and Features*. Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/what-is-mongodb-working-and-features/

Ankita_Saini. (2023). *MongoDB – Field Update Operators*. Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/mongodb-field-update-operators/

chatgpt. (2024). *chatgpt*. Retrieved from question: what is the purpose of admin, config, and local database in MongoDB: chatgpt.com

chatgpt. (2024, 2024). *Question: what does 'run service as network service user' mean?* Retrieved from chatgpt: www.chatgpt.com

dishebhhayana. (2022). *Mongoose Validation*. Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/mongoose-validation/

MongoDB CRUD Operations. (n.d.). Retrieved from MongoDB: https://www.mongodb.com/basics/crud

Ravoof, S. (2023, July). *9 Types of Mongodb Operators You Need To Know*. Retrieved from Kinsta: https://kinsta.com/blog/mongodb-operators/#:~:text=MongoDB%20Operator%20Types%201%20Logical%20Operators%20Logical%20operators,array%20operators%2C%20to%20query%20documents%20containing%20arrays.%20

Shriyan. (2021, April). *How to use mocha with mongoose ?* Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/how-to-use-mocha-with-mongoose/

zack_aayush. (2020, October). *Node.js nodemon Module*. Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/node-js-nodemon-module/